

UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Corso di Laurea in Informatica (L-31)

Fly2Meet

Progetto e prototipo di un servizio Web per incontrarsi nel mondo

Relatori

Laura Semini

Stefano Forti

Candidato

Davide Orsucci

A.A. 2022/2023

Indice

1 Introduzione **5**
- 1.1 Contesto . . . 5
- 1.2 Obiettivo della tesi . . . 7
- 1.3 Struttura della tesi . . . 7

2 Strumenti e linguaggi utilizzati **9**
- 2.1 Prolog . . . 9
  - 2.1.1 Scelta dell'implementazione . . . 11
  - 2.1.2 swiplserver . . . 12
- 2.2 Amadeus API . . . 12
  - 2.2.1 API REST . . . 13
  - 2.2.2 API di Amadeus utilizzate . . . 13
  - 2.2.3 Modalità di utilizzo . . . 15
- 2.3 Python . . . 18
  - 2.3.1 Perché Python . . . 18
  - 2.3.2 Flask . . . 18
- 2.4 HTML, CSS e JavaScript . . . 19

3 Progettazione **21**
- 3.1 Requisiti . . . 21
  - 3.1.1 Requisiti funzionali . . . 21
  - 3.1.2 Requisiti non funzionali . . . 23
- 3.2 Confronto con servizi simili . . . 23
- 3.3 Ricerca delle offerte di volo . . . 25
- 3.4 Valutazione delle soluzioni . . . 26
- 3.5 Interfaccia . . . 28
- 3.6 Rappresentazione dei fatti Prolog . . . 31
  - 3.6.1 Aeroporti . . . 31
  - 3.6.2 Itinerari OneWay . . . 31
  - 3.6.3 Itinerari che includono volo di ritorno . . . 33
  - 3.6.4 Rappresentazione delle soluzioni . . . 33

4 Implementazione **37**
4.1 Backend 37
4.1.1 Prolog 37
4.1.2 Ricerca delle soluzioni 42
4.1.3 Conversione delle offerte in fatti Prolog 44
4.2 Frontend 45
4.2.1 Ricerca delle offerte di volo 45
4.2.2 Struttura del server 49
4.2.3 Funzioni di utilità 51

5 Testing **55**
5.1 Testing Python 55
5.2 Testing Prolog 57

6 Risultati **61**
6.1 Interfaccia grafica 61
6.2 Esempi di ricerca 62

7 Conclusioni e sviluppi futuri **65**
7.1 Sommario della tesi 65
7.2 Valutazione critica del lavoro svolto 65
7.3 Sviluppi futuri 66

8 Codice Prolog **67**

Capitolo 1

Introduzione

1.1 Contesto

Il settore dei viaggi aerei è ampio: ci sono innumerevoli siti Web, agenzie di viaggio online, compagnie aeree e fornitori di servizi. Ormai è pratica comune utilizzare motori di ricerca di voli aerei per cercare, confrontare e prenotare voli aerei.

Tuttavia, il processo attraverso la quale due persone distanti possono trovare una coppia di voli aerei per incontrarsi in una città terza può essere complicato. Molte persone cercano modi efficienti per pianificare incontri lavorativi o viaggi con amici, familiari o partner che si trovano in luoghi diversi. I bisogni includono la ricerca di opzioni convenienti e la possibilità di ottenere voli che soddisfino le esigenze di entrambe le parti. Attualmente le persone che cercano di pianificare incontri devono spesso cercare manualmente le opzioni di volo da diverse fonti, confrontare i prezzi, le durate dei voli, i tempi di attesa tra l'arrivo di un volo e l'altro e vedere se le date coincidono.

Uno dei possibili processi attraverso la quale due persone tra loro distanti possono cercare una coppia di voli con una meta comune per incontrarsi potrebbe essere il seguente:

1. l'utente cerca e trova una serie di opzioni di viaggio aventi per città di partenza la prima città di partenza e per destinazione una serie di città dove vorrebbe incontrare l'altra persona (Figura 1.1),

2. l'utente cerca una serie di opzioni di viaggio aventi per città di partenza la seconda città di partenza e per destinazione le stesse città utilizzate per la prima ricerca (Figura 1.2),

3. l'utente combina le opzioni di volo aventi diversa città di partenza e per destinazione la stessa città e cerca tra queste combinazioni quella che potrebbe essere la soluzione che fa al caso loro, controllando gli orari di partenza e arrivo dei due voli aerei e il costo (Figura 1.3).

4. dopo aver scelto la coppia di voli che gli sembra più conveniente, l'utente procede alla prenotazione dei due viaggi aerei, così da potersi incontrare.

Questo processo può essere dispendioso e non tenere conto di opzioni ottimali o soluzioni creative per incontrarsi. Da queste premesse si crea l'opportunità per creare un servizio software innovativo che sia in grado di automatizzare il processo di ricerca e selezione delle coppie di voli per incontri tra persone distanti, tenendo conto delle preferenze degli utenti.

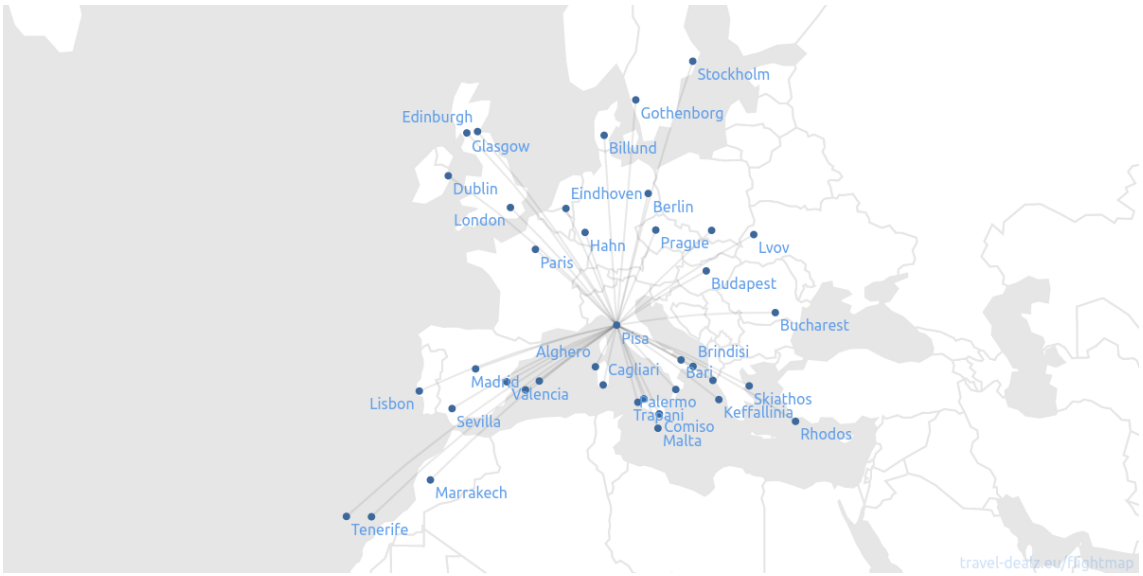


Figura 1.1: rotte aeree aventi per aeroporto di partenza l'aeroporto internazionale di Pisa (PSA).

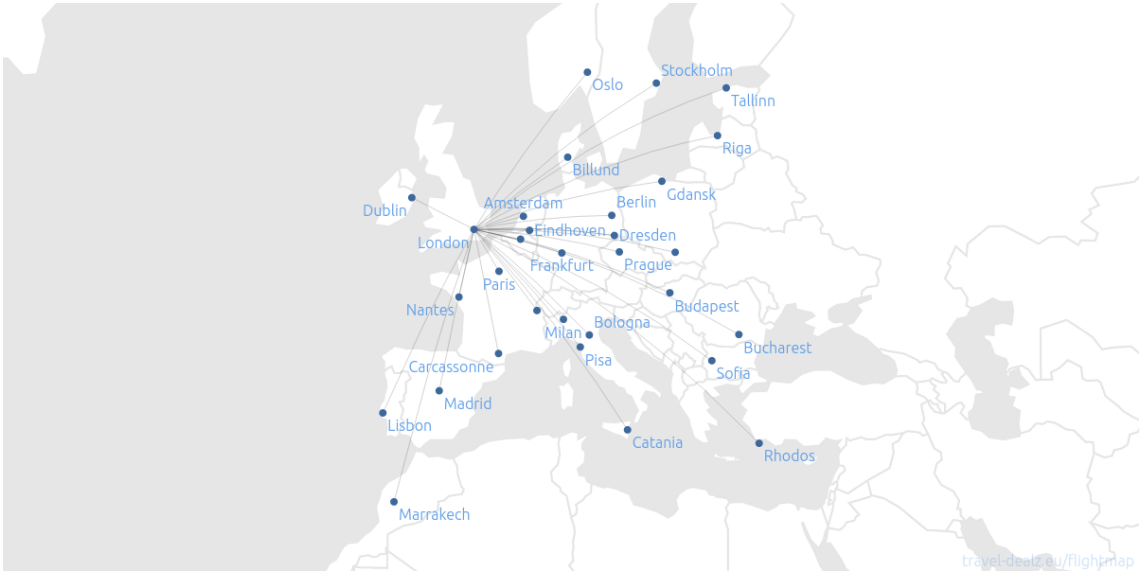


Figura 1.2: rotte aeree aventi per aeroporto di partenza l'aeroporto di Londra (LHR).

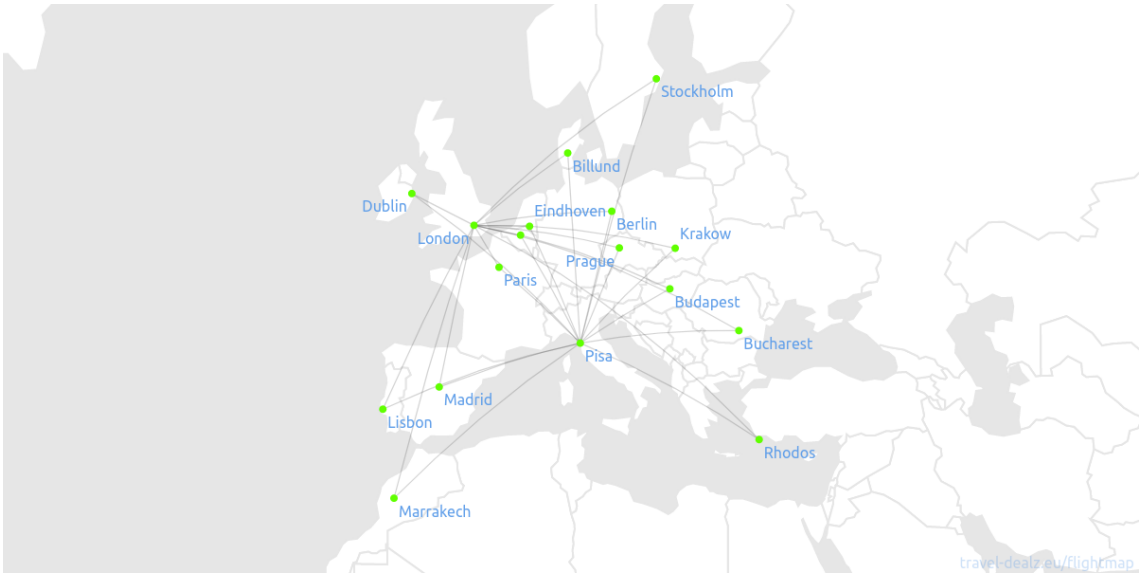


Figura 1.3: rotte aeree aventi destinazioni comuni sia all'aeroporto di Pisa che all'aeroporto di Londra.

1.2 Obiettivo della tesi

Nel contesto appena descritto:

> *L'obiettivo di questa tesi è l'implementazione del prototipo di un servizio Web utile a organizzare viaggi aerei per fare incontrare due persone provenienti da parti diverse del mondo, riducendo costi, durata dei voli e tempi di attesa una volta giunti a destinazione.*

Questa tesi si propone di sviluppare il prototipo di un sistema informatico che ha come obiettivo principale l'organizzazione di viaggi aerei per consentire a due persone provenienti da diverse parti del mondo di incontrarsi in modo efficiente. Nello specifico gli obiettivi della tesi sono:

1. *una riduzione dei costi* cercando soluzioni di viaggio che siano economicamente convenienti per entrambe le persone coinvolte, cercando offerte di volo con prezzi competitivi,

2. *una riduzione della durata dei voli* minimizzando il tempo trascorso in volo per le due persone, e

3. *una riduzione dei tempi di attesa* in modo che le persone coinvolte possano incontrarsi il prima possibile una volta giunte a destinazione.

1.3 Struttura della tesi

Il resto di questo manoscritto è organizzato come segue:

Capitolo 2 che riporta le informazioni necessarie per comprendere i capitoli seguenti relativamente alle tecnologie utilizzate durante lo sviluppo del progetto di tesi,

Capitolo 3 che discute i requisiti individuati per il progetto di tesi e, partendo da questi, mostra come è avvenuta la progettazione del sistema, dalla strategia di ricerca delle offerte di volo, passando per l'interazione con l'utente, fino al calcolo e la valutazione delle soluzioni,

Capitolo 4 che descrive come è avvenuta l'implementazione del prototipo del sistema descrivendo in maniera dettagliata come è stato scritto il codice,

Capitolo 5 che descrive come è stato effettuato il testing del prototipo del servizio Web,

Capitolo 6 che riassume i risultati ottenuti dal lavoro di tesi, e

Capitolo 7 che conclude la tesi con una valutazione critica del lavoro svolto e identifica possibili sviluppi futuri.

Capitolo 2

Strumenti e linguaggi utilizzati

In questo capitolo, si descrivono in breve gli strumenti utilizzati per lo sviluppo del progetto Fly2Meet:

- il linguaggio Prolog utilizzato per implementare la strategia di di individuazione delle combinazioni dei voli (sezione 2.1),
- la Amadeus API, necessaria per recuperare i dati sui voli (sezione 2.2),
- il linguaggio Python utilizzato per sviluppare il back-end (sezione 2.3),
- i linguaggi HTML, CSS e JavaScript utilizzati per sviluppare il front-end (sezione 2.4).

2.1 Prolog

Prolog è il più popolare linguaggio di programmazione logica in circolazione che, con una semantica e una sintassi molto semplice, permette di risolvere problemi che coinvolgono oggetti e le relazioni tra essi utilizzando un approccio dichiarativo.

Prolog utilizza il paradigma di programmazione logica emersa per la prima volta all'inizio degli anni '70 da una convergenza del lavoro nei settori della dimostrazione automatica di teoremi, intelligenza artificiale e linguaggi formali [6]. Prolog permette di risolvere problemi utilizzando un *approccio dichiarativo* alla risoluzione dei problemi: mentre in un linguaggio di programmazione imperativo ci si concentra su come risolvere un problema attraverso una serie di azioni, in Prolog ci si concentra su cosa è vero e su quali relazioni ci sono tra gli oggetti.

La programmazione in Prolog può essere riassunta in tre azioni principalmente:

1. *specificare alcuni fatti* che descrivono gli oggetti che vogliamo rappresentare e le loro relazioni. ad esempio, per definire dei fatti che rappresentano delle persone possiamo scrivere:

```
1 genitore(john,sarah).
2 genitore(john,mike).
```

```
genitore(lisa,sarah).
fratello(mike,john).
```

2. *definire alcune regole* riguardanti gli oggetti e le loro relazioni. ad esempio:

```
fratello(X,Y) :- genitore(Z,X),genitore(Z,Y),X\=Y.
zio(X,Y) :- genitore(P,Y),fratello(X,P),X\=P.
```

3. *eseguire interrogazioni* riguardanti gli oggetti e le loro relazioni.

```
?- fratello(mike,john).
true.
?- zio(X,sarah).
X = mike.
```

I predicati sono caratterizzati da una *arietà*: per riferirsi a un predicato si utilizza il suo nome seguito dal numero di argomenti che richiede (ad esempio *fratello/2*).

A differenza dell'utilizzo di approcci che prevedono la programmazione lineare intera (PLI) o di algoritmi di Machine Learning, l'approccio dichiarativo offerto da Prolog in questo contesto risulta una scelta vincente perché:

1. Prolog è un linguaggio espressivo, particolarmente adatto alla rappresentazione della conoscenza in forma di regole logiche. Sia il Machine Learning che PLI utilizzano complessi modelli matematici, il che può essere restrittivo per la rappresentazione di problemi con vincoli non lineari o relazioni complesse tra variabili.

2. Prolog permette di esprimere algoritmi complessi in modo più conciso ed elegante rispetto ad alcuni linguaggi di programmazione tradizionali. Ciò rende il codice maggiormente comprensibile, spiegabile e riduce la possibilità di errori. Molte tecniche di Machine Learning spesso vengono definite Black Box proprio perché sono difficili da interpretare [5]. Lo stesso vale per PLI dove sono richieste competenze matematiche avanzate.

3. Prolog è adatto per risolvere problemi che prevedono un'analisi logica e il ragionamento su un insieme di regole.

Prolog esegue l'inferenza automaticamente per trovare soluzioni ai problemi. Trasformando quindi in fatti Prolog ciò che vogliamo rappresentare dei viaggi aerei e scrivendo regole adatte alla risoluzione del problema è possibile trovare in maniera efficiente la soluzione ai requisiti di Fly2Meet.

Il funzionamento di Prolog si basa sul concetto di backtracking. "il backtracking consiste nel rivedere ciò che è stato fatto, tentando di soddisfare nuovamente gli obiettivi trovando un modo alternativo per soddisfarli" [1]. Prolog utilizza il backtracking per trovare tutte le possibili soluzioni a una interrogazioni o un problema specifico. Quando una interrogazioni viene eseguita in Prolog, il sistema cerca di soddisfarla trovando corrispondenze con le regole e i fatti presenti nella base di conoscenza del programma.

Prolog adotta una strategia *Depth-First Search (DFS)* per effettuare una ricerca in profondità sull'albero delle scelte e effettua il backtracking in caso di fallimento. Questa strategia di risoluzione si dice Selective Linear Definitive (SLD) [3]. Prolog utilizza questa strategia invece di una Breadth-First Search (BFS) per motivi di efficienza: poiché la DFS esplora in profondità prima di passare a nuove alternative, in genere richiede meno memoria rispetto alla BFS, che deve mantenere una coda di nodi non ancora esplorati. Inoltre le regole in Prolog possono essere fortemente annidate, con regole interne che richiamano altre regole interne. La DFS si presta meglio a gestire questo tipo di annidamento, in quanto esplora una regola fino in fondo prima di tornare indietro. Prolog potrebbe fallire in situazioni in cui ci sono problemi di ciclo infinito, ambiguità nelle scelte o errori logici nelle regole.

2.1.1 Scelta dell'implementazione

Esistono diverse implementazioni del linguaggio Prolog, ciascuna con le proprie differenze. Le versioni di Prolog prese in considerazione sono state SWI-Prolog[1] e Ciao Prolog[2]. L'implementazione utilizzata per la realizzazione di Fly2Meet è SWI-Prolog. La scelta di questa implementazione è dovuta alle seguenti motivazioni:

1. *versatilità*: SWI-Prolog è un linguaggio molto versatile e è in grado di offrire una interfaccia a alto livello verso l'esterno per i più popolari linguaggi di programmazione (di nostro particolare interesse Python) rendendolo adatto a molti contesti differenti. Ciao Prolog implementa delle estensioni avanzate rispetto al linguaggio Prolog standard come la programmazione logica con vincoli e orientata agli oggetti e un sistema di tipizzazione. Dato che l'utilizzo di un approccio puramente dichiarativo porta con sé innumerevoli vantaggi (come descritto sopra) e le estensioni avanzate per il nostro contesto non sono necessarie, viene privilegiato l'utilizzo di SWI-Prolog.

2. *prestazioni*: SWI-Prolog offre una compilazione veloce, scala bene con applicazioni di grandi dimensioni e non ha limiti alla dimensione del programma, ai valori interi e all'arietà dei fatti Prolog. Ciò lo rende una implementazione ad alte prestazioni. SWI-Prolog inoltre sfrutta il concetto di "Just-In-Time Indexing" che permette il lookup dei fatti e dei predicati presenti della base di conoscenza in manuera veloce e efficiente [7].

3. *supporto*: SWI-Prolog è open source, con una estensiva documentazione e una ampia community che offre supporto per i più diffusi sistemi operativi (Linux, Windows, Mac OS). Anche Ciao Prolog è open source, ma SWI-Prolog gode di un supporto maggiore da parte della comunità, ciò rende la sua documentazione migliore.

[1] https://www.swi-Prolog.org/
[2] https://ciao-lang.org/

4. *strumenti per lo sviluppo*: SWI-Prolog mette a disposizione strumenti e frameworks per il testing e la validazione dei programmi Prolog.



Figura 2.1: logo di SWI Prolog

2.1.2 swiplserver

Questa implementazione di Prolog risulta particolarmente efficiente da utilizzare in questo contesto perché permette di interfacciare in maniera efficiente Prolog con Python, il linguaggio di programmazione utilizzato per lo sviluppo del back-end di Fly2Meet. Per fare questo viene utilizzato il modulo *swiplserver* che «Consente di utilizzare SWI-Prolog come parte incorporata di un'applicazione, "come una libreria"» [3]. Il modulo swiplserver mette a disposizione un insieme di classi che permettono di eseguire interrogazioni SWI-Prolog in maniera trasparente e di ottenere i risultati delle interrogazioni in formato JSON. Queste classi sono:

- *PrologMQI*: gestisce automaticamente l'avvio e l'arresto di una istanza di SWI-Prolog e avvia la Machine interrogazioni Interface ('MQI') per permettere l'esecuzione di interrogazioni Prolog.

- *PrologThread*: permette di eseguire interrogazioni sul processo creato da PrologMQI. Le interrogazioni vengono eseguite esattamente come se fossero eseguite dal prompt dei comandi di Prolog. PrologThread rappresenta un thread di Prolog, e non di Python. Ciascun PrologThread è isolato dagli altri e quando viene effettuata una interrogazione su un determinato PrologThread viene sempre eseguita da quel PrologThread.

2.2 Amadeus API

Amadeus[4] (Figura 2.2) è una azienda operante nel settore dei viaggi che fornisce soluzioni tecnologiche per l'industria. In particolare, Amadeus fornisce un insieme di API che permettono di ottenere dati in tempo reale riguardo a voli, orari, prezzi e

[3]https://www.swi-Prolog.org/packages/mqi/Prologmqi.html
[4]https://developers.amadeus.com/

altro ancora. Delle tante API messe a disposizione da Amadeus utilizzeremo le Self-Service API che permettono a sviluppatori indipendenti e start-ups di connettersi e testare le API di Amadeus in maniera veloce, semplice e gratuitamente.



Figura 2.2: logo di Amadeus

2.2.1 API REST

Amadeus offre API REST (REpresentational State Transfer). Una API REST descrive un insieme di risorse e un insieme di operazioni che possono essere chiamate su quelle risorse [10].

Le API REST come quelle messe a disposizione da Amadeus si basano sui seguenti principi:

1. *Identificazione delle risorse tramite URI*: Ciascuna API di Amadeus ha un path base che serve a isolare la API dalle altre API e da altre versioni della stessa API. Concatenando al path base un path relativo è possibile accedere a una delle risorse messe a disposizione da Amadeus.

2. *Interfaccia uniforme*: Queste operazioni sulle risorse di Amadeus API possono essere effettuate da un qualsiasi client utilizzando i metodi HTTP (principalmente GET e POST).

3. *Disaccoppiamento delle risorse dalla loro rappresentazione*: si può accedere al contenuto di una risorsa in una varietà di formati differenti (e.g., HTML, XML, JSON e altri).

4. *Interazioni stateless attraverso hyperlinks*: ogni interazione con una risorsa è stateless (non mantiene lo stato).

2.2.2 API di Amadeus utilizzate

Amadeus mette a disposzione un numero consistente di API, di particolare interesse per lo svlippo di Fly2Meet sono le seguenti API:

1. *Flight Inspiration Search* [5]: l'obiettivo principale di Fly2Meet è quello di automatizzare il processo di ricerca di una meta comune per incontrarsi. Questa API risulta particolarmente adatta perché preso in ingresso il codice IATA di una città di partenza restituisce un ventaglio delle più economiche destinazioni

[5]https://developers.amadeus.com/self-service/category/flights/api-doc/flight-inspiration-search

sulla base di una determinata data di partenza e di un costo base massimo. Questa API prende per parametri:

- *origin*: è il codice IATA della città da cui intendiamo prendere il volo.
- *departureDate*: è la data nella quale il volo partirà dalla città indicata in "origin" nel formato ISO 8601 YYYY-MM-DD.
- *oneWay*: se questo parametro è impostato su true vengono considerati solo i voli di sola andata. Se questo parametro non è impostato o è impostato su false, vengono considerati solo i voli di andata e ritorno.
- *duration*: è un parametro che deve essere settato soltanto se "oneWay" è uguale a false. Indica il numero di giorni esatti di durata del viaggio.
- *nonStop*: se questo parametro ha valore true vengono considerate solo destinazioni raggiungibili con un volo diretto.
- *maxPrice*: definisce il limite di prezzo per ciascuna offerta restituita. Il valore deve essere un numero positivo, senza decimali.
- *viewBy*: permette di ordinare le destinazioni secondo un ordinamento particolare.

2. *Flight Offers Search* [6]: Questa API restituisce una lista ordinata per costo base delle soluzioni di volo più economiche con i criteri di ricerca specificati. Questa API prende i seguenti parametri:

   - *origibLocationCode*: è un parametro obbligatorio, è il codice IATA della città dalla quale si intende prendere il volo.
   - *destinationLocationCode*: è un parametro obbligatorio, è codice IATA della destinazione desiderata.
   - *departureDate*: è un parametro obbligatorio, è la data di partenza del volo nel formato ISO 8601 YYYY-MM-DD.
   - *returnDate*: è la data nella quale il viaggiatore desidera partire dalla città di destinazione per tornare alla città di origine. Se non specificata, vengono considerati soltanto itinerari con voli di sola andata.
   - *adults*: è un parametro obbligatorio, è il numero di passeggeri di età maggiore di 12 che prenderanno parte al volo. Deve essere almeno 1.
   - *children*: è il numero di passeggeri di età compresa tra 2 e 12 che prenderanno parte al volo.
   - *infants*: è il numero di passeggeri neonati la cui età è minore o uguale a 2.

[6] https://developers.amadeus.com/self-service/category/flights/api-doc/flight-offers-search

- *travelClass*: la maggior parte del tempo di volo dovrebbe essere trascorso in una cabina di questa qualità o superiore. La classe di viaggio accettata è Economy, Premium Economy, Business o First Class. Se non specificata vengono considerate tutte le classi.
- *includeAirlineCodes*: questa opzione si assicura che il sistema consideri nella ricerca solo determinate linee di volo.
- *excludedAirlineCodes*: questa opzione permette di escludere determinate linee di volo dalla ricerca.
- *nonStop*: se questa opzione ha valore true, vengono considerati soltanto voli diretti.
- *currencyCode*: è il tipo di valuta da utilizzare per la ricerca nel formato ISO 4217.
- *maxPrice*: è il prezzo massimo per viaggiatore. Per impostazione predefinita, non viene applicato alcun limite. Se specificato, il valore deve essere un numero positivo senza decimali.
- *max*: numero massimo di offerte di voli di ritorno. Se specificato, il valore deve essere maggiore o uguale a 1. Di default ha valore 250.

3. *Airport & City Search* [7]: Flight Inspiration Search e Flight Offers Search utilizzano un codice IATA di 3 lettere per effettuare la ricerca delle offerte di volo, Airport & City Search mette a disposizione un meccanismo per traslare il nome di una determinata città nel corrispettivo codice IATA, semplificando la ricerca dal lato utente. Richiede i seguenti parametri:

   - *subType*: è il tipo di posizione che si vuole ricercare (AIRPORT e/o CITY).
   - *keyword*: è una parola chiave che rappresenta l'inizio di una parola presente nel nome o nel codice di una città o di un aeroporto.
   - *view*: seleziona il livello di dettaglio nella risposta.
     - LIGHT - restituisce solo IATACode, name, detailedName, cityName e countryName.
     - FULL - aggiunge alle informazioni date da LIGHT quelle di timeZoneOffset, geocode, detailed address e travelers.score.

     l'opzione di default è FULL.

2.2.3 Modalità di utilizzo

Per accedere a una risorsa messa a disposizione Self-Service API di Amadeus il client deve:

[7] https://developers.amadeus.com/self-service/category/flights/api-doc/airport-and-city-search

1. *essere registrato*: per usufruire delle API messe a disposizione da Amadeus è necessario registrarsi sul portale di Amadeus for Developers e verificare il proprio account attraverso la email di conferma.

2. *ottenere la propria API Key e API Secret*: dal portale Amadeus for Developers è necessario ottenere dall'area riservata API Key e API Secret. Per poter accedere alle Self-Service API di Amadeus è necessario ottenere un Access Token facendo una richiesta di tipo POST al seguente endpoint

```
https://test.api.amadeus.com/v1/security/oauth2/token
```

   specificando il seguente campo data la propria API key e il proprio API secret.

```
{
    "grant_type": "client_credentials",
    "client_id": api_key,
    "client_secret": api_secret
}
```

   In risposta all'utilizzo del metodo POST si ottiene un messaggio JSON simile a quello seguente, contenente diverse informazioni tra cui proprio l'Access Token nel campo "*access_token*":

```
{
    "type": "amadeusOAuth2Token",
    "username": "d.orsucci1@studenti.unipi.it",
    "application_name": "fly2meet test",
    "client_id": "jLqpw2sD6kjN4iR6O1vXW2bOviHAPK5F",
    "token_type": "Bearer",
    "access_token": "YB4WzvpPjxMoLa3rV7kFqWn5vJ82",
    "expires_in": 1799,
    "state": "approved",
    "scope": ""
}
```

3. *effetturare una chiamata*: Nell'Header del messaggio HTTP prima di accedere a qualsiasi risorsa di Amadeus Self-Service API deve essere configurato il campo "Authorization" per contenere l'Access Token:

```
'Bearer {access_token}'
```

   Deve essere inoltre scelto il formato nel quale si vogliono ricevere le risposte, per comodità di gestione scegliamo JSON, pertanto il campo "Content-Type" dello Header deve essere

```
'application/json'
```

   Finalmente utilizzando il metodo Get o Post di HTTP su un determinato endpoint (ottenuto dalla concatenazione di base path e relative path), con i parametri necessari dell'Header e i parametri richiesti dalla API come descritto nella documentazione si può accedere alle risorse di Amadeus API.

Ad esempio alla chiamata a Flight Inspiration API per ottenere le ispirazioni di volo i parametri richiesti potrebbero essere i seguenti:

```
{
    'origin': PAR,
    'departureDate': "2022-09-06",
    'nonStop': 'true',
}
```

la risposta JSON restituita avrà una forma simile al seguente:

```
{
    "data": [
        {
            "type": "flight-destination",
            "origin": "PAR",
            "destination": "CAS",
            "departureDate": "2022-09-06",
            "returnDate": "2022-09-11",
            "price": {
                "total": "161.90"
            }
        },
        {
            "type": "flight-destination",
            "origin": "PAR",
            "destination": "AYT",
            "departureDate": "2022-10-16",
            "returnDate": "2022-10-31",
            "price": {
                "total": "181.50"
            }
        }
    ]
}
```

2.3 Python

2.3.1 Perché Python

«Python è un linguaggio di programmazione general-purpose open source; ottimizzato per la qualità, per la produttività, la portatilità e l'integrazione. Python è utilizzato da centinaia di migliaia di sviluppatori in giro per il mondo, in aree come Internet scripting, programmazione di sistemi, interfacce utente, customizzazione del prodotto e altro» [8].

Come si evince da questa definizione, Python (Figura 2.3) è un linguaggio di programmazione molto potente, che permette di adattarsi a molti contesti differenti. Uno degli utilizzi più diffusi di Python è quello di linguaggio di programmazione per lo sviluppo di applicazioni Web, che è ciò che dovrà essere Fly2Meet.

Nel contesto di Fly2Meet Python viene utilizzato per lo sviluppo del *back-end*, ossia per lo sviluppo della parte server dell'applicazione Web.



Figura 2.3: logo di Python

2.3.2 Flask

Python offre una vasta gamma di librerie e framework per lo sviluppo Web e il backend, uno di questi è Flask (Figura 2.4).



Figura 2.4: logo di Flask

Flask è un piccolo framework, «piccolo abbastanza per essere chiamato micro-framework» [4], che agevola lo sviluppo di applicazioni Web scritte in Python. Flask fornisce una strttura base e un insieme di strumenti per gestire le richieste HTTP, le risposte e altre operazioni comuni nel mondo delle applicazioni Web.

Le principali funzionalità messe a disposizione da Flask sono:

- *gestione delle richieste*: Flask semplifica la gestione delle richieste HTTP (come metodi GET e POST).

- *Routing*: Flask consente di associare determinate URL(route) a funzioni specifiche, consentendo l'indirizzamento delle richieste degli utenti alle parti appropriate dell'applicazione.

- *Template*: Flask include un motore di template chiamato *Jinja2* che permette di generare pagine HTML dinamiche in modo efficiente. Attraverso Jinja è possibile far arrivare variabili dall'applicazione Python alla pagina HTML e inserire blocchi di controllo. Ad esempio è possibile creare un array contenente i risultati utilizzando Python e successivamente è possibile visualizzarli in una interfaccia grafica. Oppure è possibile che si sia verificato un errore, in tal caso è possibile scrivere un messaggio di errore da far visualizzare all'utente.

2.4 HTML, CSS e JavaScript

Dal punto di vista del *front-end*, Fly2Meet deve presentare agli utenti pagine Web interattive e dinamiche.

Per raggiungere l'obiettivo prefissato da Fly2Meet viene utilizzata la commistione di HTML, CSS e JavaScript:

1. HTML (HyperText Markup Language) è il linguaggio di markup utilizzato per definire la struttura e il contenuto di una pagina Web.

2. CSS (Cascading Style Sheet) è un linguaggio utilizzato per definire l'aspetto e la presentazione di una pagina Web.

3. JavaScript JavaScript è un linguaggio di programmazione utilizzato per rendere le pagine Web interattive e dinamiche.

Il servizio offerto da Fly2Meet dovrà essere distribuito attraverso la sua pagina Web. In particolare verrà utilizzato un template scritto in HTML per realizzare una pagina Web dinamica, JavaScript verrà utilizzato per aggiungere dei comportamenti dinamici alla pagina Web, CSS verrà utilizzato per definire lo stile della pagina, dandogli un look e un design moderno.

Capitolo 3

Progettazione

In questo capitolo, dopo aver discusso i requisiti identificati per il sistema Fly2Meet (sezione 3.1), si descrivono la strategia di ricerca delle offerte di volo (sezione 3.3) e la funzione di ranking utile a valutare la qualità di una soluzione identificata (sezione 3.4).

Quindi, si passa alla descrizione dell'interfaccia utente, dal form di ricerca alla visualizzazione dei risultati (sezione 3.5).

Infine, viene descritta come avviene la rappresentazione delle offerte di volo e degli aeroporti per il calcolo delle soluzioni in Prolog, nonché delle soluzioni stesse (sezione 3.6).

3.1 Requisiti

In questa sezione, si elencano i requisiti, funzionali e non, raccolti per il sistema.

3.1.1 Requisiti funzionali

- Il sistema deve offrire una Web GUI che consenta agli utenti di inserire le loro preferenze di viaggio, visualizzare i risultati e interagire con il sistema in modo semplice e intuitivo.
- Il sistema deve scegliere la destinazione migliore per incontrarsi per i due utenti qualora non fosse stata specificata.
- Il sistema deve raccogliere attraverso la Web GUI le seguenti preferenze di viaggio:
  - *prima città di partenza*: è la città da cui il primo passeggero desidera partire.
  - *seconda città di partenza*: è la città da cui il secondo passeggero desidera partire.

  - *data di partenza*: è la data in cui i due passeggeri desiderano lasciare la propria città.
  - *data di ritorno*: è la data in cui i due passeggeri desiderano tornare nella propria città.
  - *costo massimo*: è il costo massimo base dichiarato dalla compagnia aerea non considerando i bagagli per i singoli voli.
  - *durata massima del viaggio*: è la durata massima del viaggio per i singoli voli.
  - *tempo di attesa massimo*: è il tempo di attesa massimo che intercorre tra l'arrivo di un passeggero e l'altro a destinazione.
  - *target cities*: se specificate, al momento della ricerca il sistema deve effettuare una ricerca avente per destinazione le target cities specificate dall'utente, altrimenti deciderà il sistema.
  - *target coutries*: se specificaie, al momento della ricerca il sistema deve effettuare una ricerca avente per destinazione i target countries specificati dall'utente, altrimenti deciderà il sistema.
  - *same airport*: città di grandi dimensioni come Parigi o Londra tendono ad avere più di un aeroporto a causa della maggiore domanda di voli e servizi aerei, i passeggeri vogliono incontrarsi nello stesso aeroporto?
  - *Explore a new city*: sono tollerate opzioni che comprendano voli diretti tra le due città oppure si vogliono visitare esclusivamente mete nuove?

- Il sistema deve considerare le limitazioni dei costi, dei tempi di volo e dei tempi di attesa e trovare le soluzioni ottimali.

- il sistema deve visualizzare i risultati all'utente in un formato chiaro e comprensibile includendo informazioni come l'elenco dei voli, aeroporti, date e tempi di partenza e arrivo, i costi dei voli, carrier code e carrier number per identificare i voli.

- Il sistema deve dare la possibilità di visualizzare i risultati di ricerca in maniera ordinata utilizzando un algoritmo di ranking delle soluzioni oppure un criterio di ordinamento scelto dall'utente (Prezzo medio, Durata media dei voli, Tempo di attesa tra l'arrivo dei due voli). Questi criteri di ordinamento sono:

  - *durata media dei voli*: la durata del volo potrebbe essere un fattore importante per entrambe le persone.
  - *tempo di attesa*: gli orari potrebbero essere essere rilevanti per sincronizzare l'incontro.
  - *prezzo medio*: il prezzo potrebbe essere un fattore importante per molte persone.

- Il sistema deve restituire le soluzioni ordinate utilizzando il proprio algoritmo di ranking come ordinamento di default.

3.1.2 Requisiti non funzionali

- Il sistema deve avere un tempo di risposta ragionevole: deve utilizzare algoritmi efficienti e tecnologie per poter soddisfare le richieste degli utenti in tempo accettabile.

- Il sistema deve essere scalabile di fronte a carico di lavoro variabile.

3.2 Confronto con servizi simili

Esistono numerosi motori di ricerca per viaggi aerei. Per effettuare un confronto ne prenderemo ad esempio due popolari che sono Volagratis.com[1] e Skyscanner.it[2]. Un modo efficace per capire le principali differenze tra Fly2Meet e servizi come Volagratis o Skyscanner è quello di mettere a confronto il loro business model. Un modo semplice e efficace per fare ciò è quello di confrontare il Business Model Canvas dei due servizi. Per effettuare il confronto può essere comodo analizzare il Business Model Canvas di Volagratis.com, il cui Business Model non si discosta molto da altri motori di ricerca per voli aerei come Skyscanner.

- *Segmenti di clientela*: Volagratis.com si rivolge principalmente a individui che cercano voli, hotel e pacchetti vacanza a prezzi convenienti.

- *Proposta di valore*: Offrire ai clienti un'ampia scelta di opzioni e la possibilità di confrontare prezzi da diverse compagnie aeree e fornitori di servizi alberghieri.

- *Canali di distribuzione*: Il principale canale di distribuzione di Volagratis.com è il suo sito Web.

- *Relazioni con i clienti*: Volagratis.com fornisce una navigazione intuitiva e opzioni di ricerca avanzate per offrire una esperienza utente positiva, nonché un servizio clienti tramite chat online, email e telefono per assistere i clienti nelle loro prenotazioni o rispondere alle domande.

- *Fonti di ricavo*: Le principali fonti di ricavo di Volagratis.com sono le commissioni guadagnate dalle compagnie aeree, dagli hotel e da altri fornitori di servizi per ogni prenotazione effettuata attraverso la piattaforma. Altre fonti di ricavo potrebbero essere offerte promozionali sponsorizzate.

- *Risorse chiave*: Le risorse chiave includono il sito Web, l'accesso al database delle compagnie aeree e delle opzioni di alloggio, il personale per il servizio clienti e il marketing per attirare nuovi clienti.

- *Partnership chiave*: Volagratis.com potrebbe stabilire partnership strategiche con compagnie aeree, fornitori di servizi alberghieri, agenzie di noleggio auto e

[1] https://www.volagratis.com/
[2] https://www.skyscanner.it/

altre aziende del settore viaggi per offrire pacchetti completi ai clienti. Inoltre, le partnership con motori di ricerca, siti di recensioni e piattaforme di confronto prezzi potrebbero aiutare a aumentare la visibilità.

- *Struttura dei costi*: I costi principali includono sviluppo e manutenzione del sito Web, spese per il personale, costi di marketing e pubblicità, commissioni da pagare alle compagnie aeree e ai fornitori, nonché eventuali costi associati all'assistenza clienti e alla gestione dei pagamenti.

Il Business Model Canvas di Fly2Meet si discosta da quello di volagratis.com e skyscanner.it in almeno 2 aspetti fondamentali:

- *Segmenti di clientela*:Fly2Meet si rivolge principalmente a persone tra loro distanti che devono incontrarsi per motivi lavorativi e ad amici e familiari geograficamente distanti che desiderano incontrarsi usufruendo di viaggi aerei.

- *Proposta di valore*: Offrire ai clienti un'ampia scelta di coppie di voli per potersi incontrare nella maniera più conveniente possibile, automatizzando il processo di ricerca e abbattendo il tempo necessario alla ricerca dei voli.

L'idea alla base di Fly2Meet si basa su approccio di *Human-centered design* [9] per la risoluzione di un problema. Questo approccio implica la comprensione del contesto di utilizzo, dei bisogni degli utenti e dei loro comportamenti per la risoluzione di un determinato problema.

Fly2Meet è un servizio innovativo che si contraddistingue in maniera significativa dai motori di ricerca di viaggi aerei sopracitati. Esistono diversi tipi di innovazione [2], in particolare Fly2Meet presenta una *innovazione di servizio* dovuta al fatto che viene semplificato il processo di scelta dei voli aerei per permettere alle persone di incontrarsi. Inoltre Fly2Meet è una *innovazione di prodotto*: nessun altro motore di ricerca per voli aerei offre la possibilità di automatizzare la scelta delle coppie di viaggi aerei.

Le principali innovazioni Fly2Meet sono le seguenti:

1. *Innovare il settore dei viaggi*: Fly2Meet rappresenta una innovazione nel modo in cui le persone pianificano e coordinano i loro viaggi. L'obiettivo principale di Fly2Meet è quello di semplificare e ottimizzare il processo di ricerca delle coppie di voli, riducendo lo stress e il tempo necessario per prendere decisioni.

2. *Automazione dei processi*: L'innovazione risiede nell'automatizzazione del processo di ricerca e selezione delle coppie di voli, prendendo in considerazione le preferenze di volo, il costo medio, la durata media dei voli e il tempo di attesa che intercorre tra l'arrivo dei due voli. Questo viene fatto attraverso l'uso di algoritmi di ranking appositi.

3. *Efficienza e risparmio di tempo*: Fly2Meet mira a far risparmiare tempo agli utenti, eliminando la necessità di cercare manualmente le opzioni di volo, con-

frontare i prezzi e controllare altri fattori che influiscono sull'esperienza di viaggio. Rendere il processo di ricerca delle coppie di voli meno complicato e più efficiente potrebbe migliorare la User Experience.

4. *Servizio personalizzato*: Fly2Meet come opzione predefinita sceglie e ordina le coppie di voli attraverso il proprio algoritmo di ranking oppure sulla base di un ordinamento richiesto dall'utente: media dei costi tra i due viaggi aerei, media delle durate dei due viaggi aerei oppure tempo di attesa trai due voli aerei.

3.3 Ricerca delle offerte di volo

Se l'utente non specifica alcuna città o paese di preferenza, deve essere Fly2Meet a cercarle per lui. La ricerca manuale dei voli diretti verso ogni possibile destinazione è un'operazione dispendiosa, le API messe a disposizione da Amadeus però ottimizzare notevolmente il processo di ricerca. La procedura di ricerca dei voli utilizzata da Fly2Meet è la seguente:

1. *Conversione dei nomi delle città in codici IATA*: Le API di Amadeus utilizzano per parametri codici IATA di 3 lettere maiuscole, è necessario quindi fare una conversione delle due città di partenza(e.g. 'Paris' deve essere convertito in 'PAR', 'London' in 'LON'). Viene utilizzata la *Airport & City Search* per fare questa conversione.

2. *Ricerca delle ispirazioni di volo*: Per restringere l'insieme di tutte le destinazioni possibili si utilizza l'API di Amadeus *Flight inspiration Search* utilizzando per parametro il codice IATA della prima città. Il risultato sarà l'insieme dei codici IATA delle destinazioni più economiche per la prima città nella data specificata e al prezzo massimo base eventualmente specificato dall'utente.

3. *Ricerca dei voli per la prima città*: Per ogni destinazione trovata dalla chiamata a Flight inspiration Search bisogna eseguire una ricerca dei voli disponibili aventi per parametri il codice IATA della prima città di partenza e il codice IATA della città di destinazione, nonché gli altri filtri di ricerca decisi dall'utente. Per fare questo si utilizza la API *Flight Offers Search* che permette di trovare le offerte di volo più economiche.

4. *Ricerca dei voli per la seconda città*: In maniera simile alla ricerca delle offerte di volo per la prima città. Viene effettuata una ricerca delle offerte di volo aventi per codice IATA della città di partenza il codice della seconda città e per destinazione le stesse destinazioni utilizzate dalla prima città di partenza.

Se l'utente ha specificato almeno una città come punto di incontro preferito, invece, la ricerca viene fatta nel seguente modo:

1. *Conversione dei nomi in codici IATA*: viene fatta la conversione dei nomi delle città di partenza in codici IATA. Deve essere inoltre fatta la conversione delle città di destinazione indicate dall'utente.

2. *Ricerca delle offerte di volo per la prima città*: si effettuano usando *Flight Offer Search* e usando per destinazione i codici IATA delle città indicate dall'utente.

3. *Ricerca delle offerte di volo per la seconda città*: come per la prima città, soltanto che il codice IATA della città di partenza è della seconda città.

Se l'utente ha specificato almeno un paese come paese di incontro preferito, la ricerca viene fatta con questa procedura:

1. *Conversione dei nomi in codici IATA*: viene fatta la conversione dei nomi delle città di partenza in codice IATA.

2. *Ricerca delle offerte di volo*: viene effettuata una ricerca delle offerte di volo utilizzando per parametri come codici IATA di partenza di quello calcolato dalla conversione e come codice IATA di destinazione quello dei principali aeroporti dei paesi specificati dall'utente.

3.4 Valutazione delle soluzioni

Dopo aver trovato tutte le offerte di volo riferite alla prima e alla seconda città di partenza, il sistema calcola tutte le possibili coppie di voli tra loro compatibili che possono essere soluzioni al problema. Due coppie di voli affinché siano compatibili devono avere stessa destinazione e tempo di attesa, prezzo base e durata non superiore a quelli indicati dall'utente. Il sistema deve però anche dare un ordinamento a queste soluzioni, privilegiando quelle che potrebbero essere di maggiore gradimento per l'utente. Il sistema deve quindi utilizzare una funzione per valutare una determinata soluzione.

Attraverso questa funzione è possibile quindi dare un punteggio alle soluzioni. Il calcolo del ranking avviene nel seguente modo:

1. Si calcolano i valori da assegnare alle variabili da inserire nella funzione euristica per il calcolo del rank. Per ciascuna coppia di voli ammissibile trovata si procede calcolando costo medio in euro, durata media di viaggio in secondi e il tempo di attesa in secondi che intercorre tra l'arrivo dei due voli a destinazione:

$$AvgPrice \longleftarrow \frac{Price1+Price2}{2}$$
$$AvgDuration \longleftarrow \frac{Duration1+Duration2}{2}$$
$$WT \longleftarrow |Arrival1 - Arrival2|$$

2. i valori trovati al punto precedente appartengono a scale di grandezza differente. Portarli sulla stessa scala di misura permette a ciascuna variabile di contribuire in modo equo alla funzione euristica, indipendentemente dalla loro scala iniziale. Per fare ciò occorre effettuare una *normalizzazione* dei valori. La normalizzazione avviene nel seguente modo:

$$NormPrice \longleftarrow \frac{AvgPrice - MinPrice}{MaxPrice - MinPrice}$$
$$NormDuration \longleftarrow \frac{AvgDuration - MinDuration}{MaxDuration - MinDuration}$$
$$NormWT \longleftarrow \frac{WT}{UserWT}$$

dove $MaxPrice$ e $MinPrice$ sono costo massimo e minimo dei voli, $MaxDuration$ e $MinDuration$ sono durata massima e minima di volo da ricercare confrontando tutte le offerte di volo. $UserWT$ è il tempo di attesa massimo desiderato dall'utente (di default infinito).

3. si utilizza la funzione euristica per il calcolo del ranking:

   $Rank \longleftarrow wPrice \cdot NormPrice + wDuration \cdot NormDuration + wWT \cdot NormWT$

   dove $wPrice$, $wDuration$ e wWT sono i pesi da associare a ciascuna variabile normalizzata e la cui somma deve valere 1. Nel nostro caso dato che vogliamo che le soluzioni migliori riescano a bilanciare costo, durata e tempo di attesa opteremo per avere lo stesso peso per tutte e tre le variabili, quindi:

$$wPrice = wDuration = wWT = \frac{1}{3}.$$

Con la strategia di ricerca dei voli descritta in 3.3 e la funzione di valutazione delle soluzioni è possibile definire la procedura attraverso la quale avviene la l'individuazione dei voli, che è quella descritta nella figura 3.1.

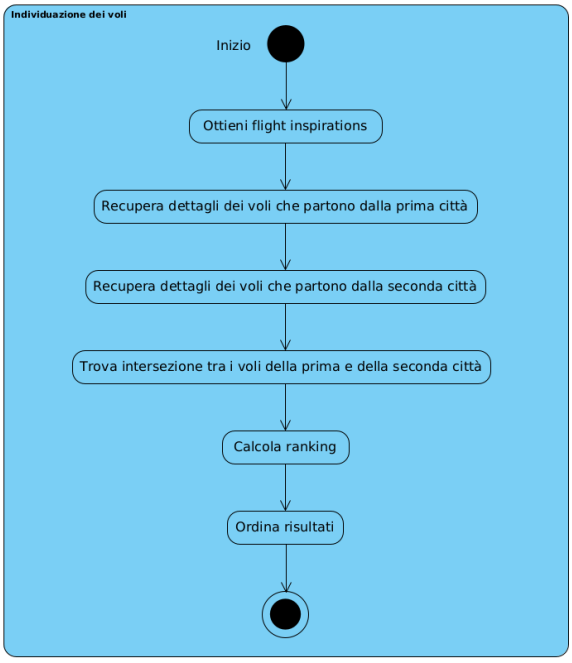


Figura 3.1: Diagramma di attività ad alto livello delle attività svolte per l'individuazione dei voli

3.5 Interfaccia

L'interazione col sistema avviene attraverso una *Web GUI* dopo aver caricato la pagina principale di Fly2Meet attraverso browser Web. La Web gui fornisce una interfaccia utente che mette a disposizione pulsanti e campi di input per accogliere le richieste dell'utente e visualizzare i risultati della ricerca. La Web GUI mette a disposizione un form per la ricerca minimale per effettuare ricerche veloci (Figura 3.2). Questo form può essere espanso dando la possibilità di mostrare delle opzioni di ricerca avanzate (Figura 3.3). Nella versione semplice del form di ricerca 3.2 vengono raccolti:

1. *Departure City 1*: nome della prima città di partenza
2. *Departure City 2*: nome della seconda città di partenza
3. *Departure Date*: data di partenza
4. *Return Date*: data di ritorno nell'eventualità in cui si vuole specificare anche il volo di ritorno.

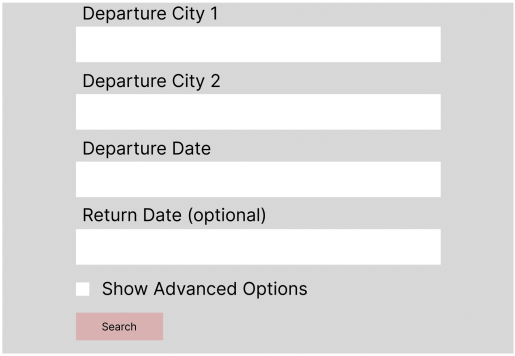


Figura 3.2: Wireframe dell'interfaccia minimale. Quando l'utente accede al sito vedrà questo form senza filtri di ricerca avanzati.

La versione avanzata del form di ricerca 3.3 mostra i seguenti filtri di ricerca opzionali:

1. *Max Base Price*: prezzo base massimo che si ha intenzione di pagare per ciascun volo
2. *Max Duration*: durata massima di ciascun volo (in ore).
3. *Max Wait Time*: tempo di attesa massimo tra l'arrivo dei due voli in minuti, di default infinito.
4. *Target Cities*: lista delle città in cui si desidererebbe incontrarsi, separate da una virgola.

5. *Target Countries*: lista dei paesi in cui si desidererebbe incontrarsi, separati da virgola.

6. *Explore a new city*: flag per indicare la volontà di non ammettere voli città 1-città 2 (e viceversa) nella soluzione, dando la possibilità di scoprire nuovi posti.

7. *Same Airport*: flag per indicare se si desidera che i due voli abbiano lo stesso aeroporto per incontrarsi.

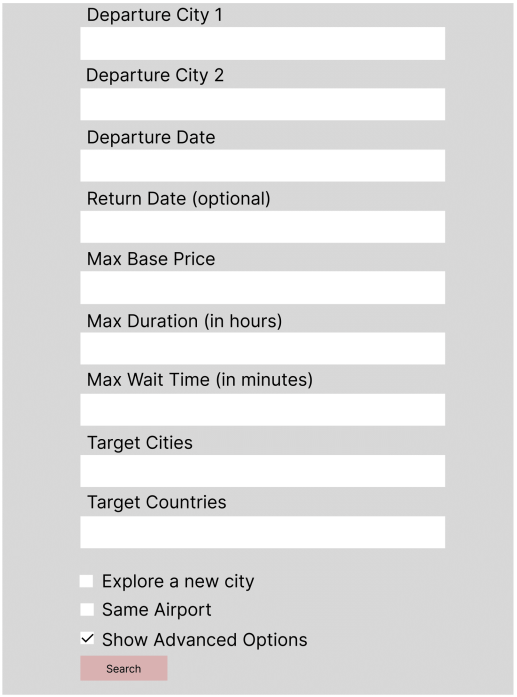


Figura 3.3: Wireframe dell'interfaccia avanzata. L'utente dopo aver scelto l'opzione "Show Advanced Options" vede dei filtri di ricerca avanzati.

Dopo aver premuto il pulsante "Search", il sistema cerca le soluzioni utilizzando i parametri dell'utente. Se viene inserita solo la data di partenza le soluzioni mostrate sono coppie di voli di sola andata verso la comune destinazione(Figura 3.4). Nel caso in cui venga inserita anche una data di ritorno le soluzioni presenteranno informazioni riguardanti anche i voli di ritorno (Figura 3.5). Ciascuna soluzione mostra:

- codice IATA degli aeroporti di partenza e arrivo per ciascun segmento degli itinerari di volo.

- orario di partenza e arrivo di entrambi i voli.

- carrier code e carrier number di ciascun volo aereo

- prezzo base di ciascun itinerario della soluzione

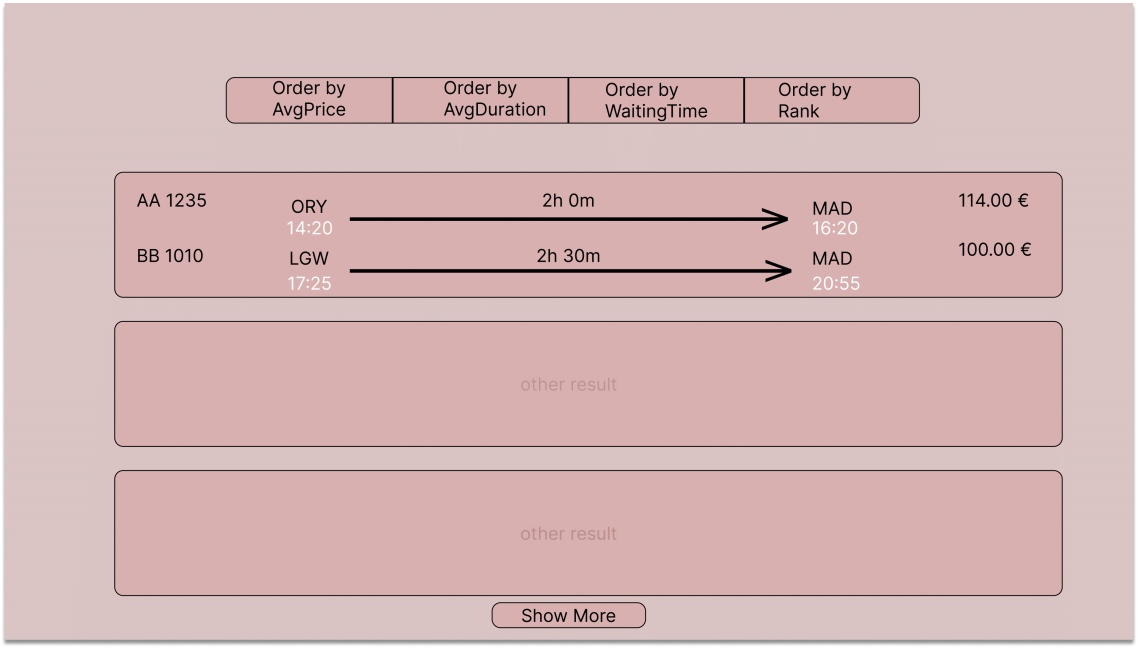


Figura 3.4: wireframe che mostra come avviene la visualizzazione dei risultati.

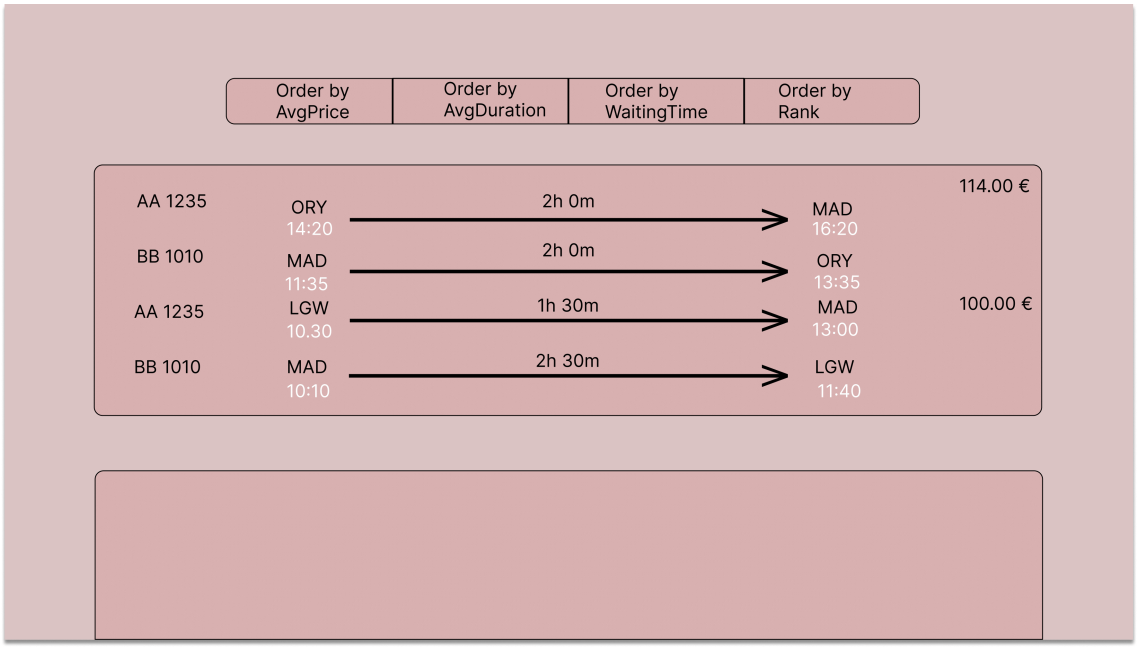


Figura 3.5: wireframe che mostra come avviene la visualizzazione dei risultati se è stata immessa anche una data di ritorno.

3.6 Rappresentazione dei fatti Prolog

Flight Offer Search di Amadeus permette di cercare delle offerte di volo utilizzando dei filtri di ricerca e di ottenerle in formato JSON. Per funzionare il nostro programma Prolog per il calcolo delle soluzioni deve utilizzare fatti Prolog che rappresentino le offerte di volo e gli aeroporti.

3.6.1 Aeroporti

Gli aeroporti vengono rappresentati con fatti del tipo:

```
airport(IATA, City, Country).
```

dove:

1. *IATA* è il codice IATA di un determinato aeroporto con lettere minuscole(e.g. l'aeroporto di Madrid–Barajas viene rappresentato come *mad*, l'aeroporto di Parigi Orly viene rappresentato come *ory*).
2. *City* è una rappresentazione testuale del nome della città a cui appartiene l'aeroporto (e.g. 'Madrid', 'Paris')
3. *Country* è la rappresentazione nel formato *ISO 3166-1 alpha-2* del paese di appartenenza dell'aeroporto (e.g. Spagna è *es*, Francia è *fr*).

   Esempi di rappresentazione degli aeroporti in fatti Prolog:

```
airport(mad,'Madrid',es).
airport(ory,'Paris',fr).
airport(lgw,'London',gb).
airport(lhr,'London',gb).
airport(cdg,'Paris',fr).
```

3.6.2 Itinerari OneWay

La rappresentazione per i voli OneWay (che non includono volo di ritorno) utilizzano fatti della forma:

```
flight(Origin, Destination, CarrierCode, FlightNumber, Departure,
    Arrival, Duration, Price).
```

dove:

1. *Origin* è il codice IATA dell'aeroporto di partenza.
2. *Destination* è il codice IATA dell'aeroporto di destinazione.
3. *CarrierCode* è un codice di due lettere utilizzato per identificare univocamente una compagnia aerea.
4. *FlightNumber* è un codice per identificare univocamente un volo aereo.

5. *Departure* è la data di partenza del volo aereo dall'aeroporto di origine.
6. *Arrival* è la data di arrivo del volo aereo all'aeroporto di destinazione.
7. *Duration* è la durata del volo in secondi.
8. *Price* è il costo del volo aereo.

per la rappresentazione di *Departure* e *Arrival* viene utilizzato la struttura dati *date-time* di SWI Prolog [3]:

```
date(Y, M, D, H, Mn, S, Off, TZ, DST)
```

dove:

1. *Y*: rappresenta l'anno,
2. *M*: rappresenta il mese,
3. *D*: rappresenta il giorno,
4. *H*: rappresenta l'ora,
5. *Mn*: rappresenta i minuti,
6. *S*: rappresenta i secondi,
7. *Off*: rappresenta l'offset in secondi rispetto a UTC. Valori positivi sono a Ovest di Greenwich,
8. *TZ*: rappresenta il nome della timezone locale. Se la timezone è sconosciuta il valore da assegnare è l'atomo -, e
9. *DST*: rappresenta un valore booleano che ha valore di true se l'ora legale è da applicare e false altrimenti. Se DST è sconosciuto o se non c'è l'ora legale allora DST è l'atomo -.

Esempi di rappresentazione di *flight/8* sono i seguenti:

```
flight(cdg, mad, '6x', 3868, Departure, Arrival, 8100, 167.59),
Departure is date(2023,8,21,16,45,0,3600,'cest',true),
Arrival is date(2023,8,21,19,0,0,3600,'cest',true).
```

[3] https://www.swi-prolog.org/pldoc/man?section=timedate

3.6.3 Itinerari che includono volo di ritorno

Per la rappresentazione delle offerte di volo che includono anche un volo di ritorno si utilizzano fatti nella forma:

```
itinerary(Origin, Destination, OutboundFlight, ReturnFlight,
    Price).
```

dove *OutboundFlight* e *ReturnFlight* sono termini composti aventi la seguente forma:

```
f(CarrierCode,FlightNumber,Departure,Arrival,Duration).
```

Un esempio di rappresentazione di *itinerary/5* è:

```
itinerary(ory, mad, OutboundFlight, ReturnFlight, 129.81),
outboundFlight is f('ib', 3417, date(2023, 8, 22, 7, 30, 0, 3600,
    'cest', true), date(2023, 8, 22, 9, 30, 0, 3600, 'cest', true
    ), 7200),
ReturnFlight is f('ib', 3402, date(2023, 9, 4, 7, 30, 0, 3600, '
    cest', true), date(2023, 9, 4, 9, 30, 0, 3600, 'cest', true),
    7200).
```

3.6.4 Rappresentazione delle soluzioni

Ciascuna soluzione deve essere caratterizzata da un rank che indica quanto e dalle informazioni sui voli che compongono la soluzione. Per coppie di voli di sola andata le soluzioni sono rappresentate con fatti della forma:

```
sol(AvgPrice, AvgDuration, WaitingTime, Rank, Flight1Info,
    Flight2Info).
```

dove:

- *AvgPrice* è la media di prezzo dei due voli,
- *AvgDuration* è la media delle durate dei due voli,
- *WaitingTime* è il tempo di attesa,
- *Rank* è il rank calcolato attraverso la funzione di valutazione,
- *Flight1Info* sono le informazioni del primo volo di andata della coppia, e
- *Flight2Info* sono le informazioni il secondo volo di andata della coppia.

Se nella ricerca non è stata scelta l'opzione *Explore a new city*, potrebbero apparire soluzioni che prevedono un volo diretto tra le due città. In tal caso la soluzione avrà la forma:

```
sol(AvgPrice, AvgDuration, WaitingTime, Rank, Flight1Info, _).
```

esempi di soluzione:

```
sol(132.22500000000002, 6750, 900.0, 0.2460788521675149, f(cdg,
    mad, ib, 3743, date(2023, 8, 21, 7, 0, 0, 3600, cest, true),
    date(2023, 8, 21, 9, 15, 0, 3600, cest, true), 8100, 64.59), f
    (lhr, mad, '6x', 9931, date(2023, 8, 21, 10, 30, 0, 0, bst,
    true), date(2023, 8, 21, 13, 0, 0, 3600, cest, true), 5400,
    199.86)).

sol(106.725, 8400, 600.0, 0.356483809212364, f(cdg, mad, ib,
    3741, date(2023, 8, 21, 16, 50, 0, 3600, cest, true), date
    (2023, 8, 21, 19, 0, 0, 3600, cest, true), 7800, 92.59), _).
```

Se è stata scelta anche una data di ritorno desiderata durata la ricerca, le soluzioni devono prevedere due voli di andata e due voli di ritorno. In questo caso la soluzione sarà della forma:

```
sol(AvgPrice, AvgDuration, WaitingTime, Rank, OutboundFlight1,
    ReturnFlight1, OutboundFlight2, ReturnFlight2).
```

dove:

- *AvgPrice* è la media di prezzo dei due voli,
- *AvgDuration* è la media delle durate dei due voli,
- *WaitingTime* è il tempo di attesa,
- *Rank* è il rank calcolato attraverso la funzione di valutazione,
- *OutboundFlight1* sono le informazioni del volo di andata del primo itinerario,
- *ReturnFlight1* sono le informazioni del volo di ritorno del secondo itinerario,
- *OutboundFlight2* sono le informazioni del volo di andata del secondo itinerario,
- *ReturnFlight2* sono le informazioni del volo di ritorno del secondo itinerario.

Qualora fosse stata scelta l'opzione *Explore a new city* durante la ricerca, potrebbero apparire soluzioni che prevedono un volo diretto tra le due città e un volo di ritorno. In tal caso la soluzione avrà la forma:

```
sol(AvgPrice, AvgDuration, WaitingTime, Rank, OutboundFlight1,
    ReturnFlight1, _, _).
```

esempi di soluzioni sono i seguenti:

```
sol(174.60500000000002, 6750, 900.0, 0.30635295337178187, f(cdg,
    mad, ib, 3743, date(2023, 8, 22, 7, 0, 0, 3600, cest, true),
    date(2023, 8, 22, 9, 15, 0, 3600, cest, true), 8100, 130.97),
    f(mad, cdg, ib, 3402, date(2023, 9, 4, 7, 30, 0, 3600, cest,
    true), date(2023, 9, 4, 9, 30, 0, 3600, cest, true), 7200,
    130.97), f(lhr, mad, '6x', 9931, date(2023, 8, 22, 10, 30, 0,
    0, bst, true), date(2023, 8, 22, 13, 0, 0, 3600, cest, true),
    5400, 218.24), f(mad, lhr, '6x', 459, date(2023, 9, 4, 10, 5,
    0, 3600, cest, true), date(2023, 9, 4, 11, 30, 0, 0, bst, true
    ), 8700, 218.24)).
```

```
2
3 sol(165.60500000000002, 8400, 600.0, 0.3238167037330218, f(cdg,
     mad, ib, 3741, date(2023, 8, 22, 16, 50, 0, 3600, cest, true),
      date(2023, 8, 22, 19, 0, 0, 3600, cest, true), 7800, 144.97),
      f(mad, cdg, ib, 3402, date(2023, 9, 4, 7, 30, 0, 3600, cest,
     true), date(2023, 9, 4, 9, 30, 0, 3600, cest, true), 7200,
     144.97), _, _)
```

Capitolo 4

Implementazione

In questo capitolo viene mostrato e descritto il codice che è stato prodotto per realizzare il prototipo del servizio utilizzando gli strumenti descritti nel capitolo 2 e le strategie di progettazione descritte nel capitolo 3. Il capitolo utilizza un approccio bottom-up per la descrizione l'implementazione del software, descrivendo prima il *backend* e poi il *frontend*. Il capitolo è composto come segue:

Sezione 4.1 descrive come è stato realizzato il backend di Fly2Meet, partendo dalla descrizione del codice Prolog utilizzato per trovare le soluzioni(Sezione 4.1.1), passando per la descrizione dell'interfaccia tra Prolog e Python per trovare le soluzioni (Sezione 4.1.2) e infine la descrizione del parser utilizzato per convertire le offerte di volo dal formato JSON in fatti Prolog (Sezione 4.1.3), e

Sezione 4.2 descrive come è stato realizzato il frontend, descrivendo il processo della ricerca delle offerte di volo attraverso Amadeus API (Sezione 4.2.1) e come è stato organizzato il Server per raccogliere le richieste degli utenti (Sezione 4.2.2).

L'implementazione completa del sistema è possibile trovarla nella repository GitHub di Fly2Meet[1].

4.1 Backend

4.1.1 Prolog

In questa sezione viene descritto il codice Prolog utilizzato per cercare le soluzioni. Il codice completo è possibile trovarlo in appendice nel capitolo 8.

Le soluzioni vengono cercate utilizzando il predicato Prolog *fly2meet/8* che cerca le offerte di volo all'interno della knowledge base utilizzando i filtri di ricerca dell'untente. Le soluzioni vengono restituite con un ordinamento indicato in *SortStrategy*. Esistono due versioni del predicato fly2meet/8. La prima utilizza il flag NewCity di valore "yes" ed è strutturata in questo modo:

[1]https://github.com/DadeOrsu/Fly2Meet

```
1  fly2meet(Airport1, Airport2, SortStrategy, Return, CapWT,
       SameAirport, yes, Flights) :-
2      airport(Airport1, _, _), airport(Airport2, _, _), dif(
           Airport1, Airport2),
3      minAndMax(price, Return, MaxP, MinP),
4      minAndMax(duration, Return, MaxD, MinD),
5      findall(FCombo, eligibleFlights(Airport1, Airport2, MaxP,
           MinP, MaxD, MinD, Return, CapWT, SameAirport, FCombo),
           TmpFlights),
6      sortByStrategy(TmpFlights, SortStrategy, Flights).
```

dove:

alla riga 2 controlla che i due codici IATA inseriti siano di aeroporti presenti nella knowledge base e che siano tra loro differenti,

alla riga 3 e 4 vengono cercati *MaxP* e *MaxP* che sono rispettivamente il costo massimo e la durata massima dei voli all'interno della base di conoscenza,

alla riga 5 il predicato *findall/3* trova tutte le possibili soluzioni composte da due itinerari di volo e aventi per destinazione una terza città. I risultati contenuti all'interno di *TmpFlights*, e

alla riga 6 vengono ordinati secondo il criterio descritto dalla *SortStrategy*.

La seconda versione di *fly2meet/8* con valore NewCity "no" è strutturata in questo modo:

```
7  fly2meet(Airport1, Airport2, SortStrategy, Return, CapWT,
       SameAirport, no, Flights) :-
8      airport(Airport1, _, _), airport(Airport2, _, _), dif(
           Airport1, Airport2),
9      minAndMax(price, Return, MaxP, MinP),
10     minAndMax(duration, Return, MaxD, MinD),
11     findall(FCombo, eligibleFlights(Airport1, Airport2, MaxP,
           MinP, MaxD, MinD, Return, CapWT, SameAirport, FCombo),
           TmpFlights1),
12     findall(FCombo, directFlight(Airport1, Airport2, MaxP, MinP,
           MaxD, MinD, Return, FCombo), TmpFlights2),
13     findall(FCombo, directFlight(Airport2, Airport1, MaxP, MinP,
           MaxD, MinD, Return, FCombo), TmpFlights3),
14     append([TmpFlights1,TmpFlights2,TmpFlights3], TmpFlights),
15     sortByStrategy(TmpFlights, SortStrategy, Flights).
```

dove in aggiunta alla prima versione:

alla riga 12 viene usato *findall/3*per cercare i voli con origine la prima città e destinazione la seconda città,

alla riga 13 viene usato *findall/3*per cercare i voli con origine la seconda città e destinazione la prima città, e

alla riga 14 vengono concatenati le liste contenenti i risultati intermedi in *TmpFlights*.

Il predicato *eligibleFlights/10* trova una possibile soluzione per Fly2Meet. Se il flag *SameAirport* ha valore "yes" gli itinerari di volo che compongono la soluzione avranno lo stesso aeroporto di destinazione. Se il flag ha valore "no" la coppia di voli che compone la soluzione potrebbe avere per destinazioni due aeroporti differenti, ma sempre nella stessa città. Il predicato *append/3* alla riga 14 concatena i 3 Array contenenti i risultati intermedi e successivamente li ordina alla riga 15 sulla base del valore di *SortStrategy*.

```
16 eligibleFlights(Airport1, Airport2, MaxP, MinP, MaxD, MinD,
       Return, CapWT, yes, Solution):-
17     airport(Airport3, _, _), dif(Airport1, Airport3), dif(
           Airport2, Airport3),
18     solutionAndRank(Airport1, Airport2, Airport3, Airport3, MaxP,
           MinP, MaxD, MinD, Return, CapWT, Solution).
19
20 eligibleFlights(Airport1, Airport2, MaxP, MinP, MaxD, MinD,
       Return, CapWT, no, Solution):-
21     airport(Airport3, City, _), airport(Airport4, City, _),
22     solutionAndRank(Airport1, Airport2, Airport3, Airport4, MaxP,
           MinP, MaxD, MinD, Return, CapWT, Solution).
```

I predicati *eligibleFlights/10* utilizzano alla riga 18 e 22 il predicato *solutionAndRank* che trovano una coppia di itinerari di volo compatibili e il rank della soluzione trovata utilizzando il predicato *solutionAndRank* alla riga 18 e 22. Il predicato *solutionAndRank* è definito in due versioni:

```
23 solutionAndRank(Airport1, Airport2, Airport3, Airport4, MaxP,
       MinP, MaxD, MinD, no, CapWT, Solution) :-
24     flight(Airport1, Airport3, CarrierNo13, FlightNo13, DepDate13
           , ArrDate13, Duration13, Price13),
25     Flight1Info=f(Airport1, Airport3, CarrierNo13, FlightNo13,
           DepDate13, ArrDate13, Duration13, Price13),
26     flight(Airport2, Airport4, CarrierNo24, FlightNo24, DepDate24
           , ArrDate24, Duration24, Price24),
27     Flight2Info=f(Airport2, Airport4, CarrierNo24, FlightNo24,
           DepDate24, ArrDate24, Duration24, Price24),
28     avg(Price13,Price24,AvgPrice),
29     avg(Duration13,Duration24,AvgDuration),
30     waitingTime(ArrDate13, ArrDate24, WaitingTime), WaitingTime
           =< CapWT,
31     travelScore(AvgPrice,AvgDuration,WaitingTime,MinP,MaxP,MinD,
           MaxD,CapWT,Rank),
32     Solution=sol(AvgPrice, AvgDuration, WaitingTime, Rank,
           Flight1Info, Flight2Info).
33
34 solutionAndRank(Airport1, Airport2, Airport3, Airport4, MaxP,
       MinP, MaxD, MinD, yes, CapWT, Solution) :-
35     itinerary(Airport1, Airport3, OutboundFlight13,
           ReturnFlight13, Price13),
```

```
36      itinerary(Airport2, Airport4, OutboundFlight24,
            ReturnFlight24, Price24),
37      OutboundFlight13 = f(CarrierNo13, FlightNo13, DepDate13,
            ArrDate13, Duration13),
38      OutboundFlight24 = f(CarrierNo24, FlightNo24, DepDate24,
            ArrDate24, Duration24),
39      ReturnFlight13 = f(CarrierNo31, FlightNo31, DepDate31,
            ArrDate31, Duration31),
40      ReturnFlight24 = f(CarrierNo42, FlightNo42, DepDate42,
            ArrDate42, Duration42),
41      avg(Price13, Price24, AvgPrice),
42      avg(Duration13, Duration24, AvgDuration),
43      waitingTime(ArrDate13, ArrDate24, WaitingTime), WaitingTime
            =< CapWT,
44      travelScore(AvgPrice,AvgDuration,WaitingTime,MinP,MaxP,MinD,
            MaxD,CapWT,Rank),
45      Flight1Info = f(Airport1, Airport3, CarrierNo13, FlightNo13,
            DepDate13, ArrDate13, Duration13, Price13),
46      Flight1ReturnInfo = f(Airport3, Airport1, CarrierNo31,
            FlightNo31, DepDate31, ArrDate31, Duration31, Price13),
47      Flight2Info = f(Airport2, Airport4, CarrierNo24, FlightNo24,
            DepDate24, ArrDate24, Duration24, Price24),
48      Flight2ReturnInfo = f(Airport4, Airport2, CarrierNo42,
            FlightNo42, DepDate42, ArrDate42, Duration42, Price24),
49      Solution=sol(AvgPrice, AvgDuration, WaitingTime, Rank,
            Flight1Info, Flight1ReturnInfo, Flight2Info,
            Flight2ReturnInfo).
```

dove:

- Se il flag *Return* vale "no" la soluzione da cercare è composta da voli di sola andata, quindi cerca una coppia di predicati *flight/8*, e
- Se il flag vale "yes" la soluzione è da cercare comprende anche un volo di ritorno, quindi si cerca una coppia di predicati *itinerary/5*.

Per trovare i voli diretti tra le due città si utilizza il predicato *directFlight/8* che cerca una soluzione avente itinerario di volo diretto tra le due città e ne calcola il rank. Anche qua se il flag *Return* ha valore "no" viene cercato un predicato *itinerary/5*.

```
50  directFlight(Airport1, Airport2, MaxP, MinP, MaxD, MinD, no,
        Solution) :-
51      flight(Airport1, Airport2, CarrierNo, FlightNo, DepDate,
            ArrDate, Duration, Price),
52      travelScore(Price, Duration, 0, MinP, MaxP, MinD, MaxD, inf,
            Rank),
53      Solution=sol(Price, Duration, 0, Rank, f(Airport1, Airport2,
            CarrierNo, FlightNo, DepDate, ArrDate, Duration, Price), _
            ).
54
55  directFlight(Airport1, Airport2, MaxP, MinP, MaxD, MinD, yes,
        Solution) :-
```

```
56      itinerary(Airport1, Airport2, OutboundFlight12,
            ReturnFlight12, Price12),
57      OutboundFlight12 = f(CarrierNo12, FlightNo12, DepDate12,
            ArrDate12, Duration12),
58      ReturnFlight12 = f(CarrierNo21, FlightNo21, DepDate21,
            ArrDate21, Duration21),
59      Flight1Info=f(Airport1, Airport2, CarrierNo12, FlightNo12,
            DepDate12, ArrDate12, Duration12, Price12),
60      Flight1ReturnInfo = f(Airport2, Airport1, CarrierNo21,
            FlightNo21, DepDate21, ArrDate21, Duration21, Price12),
61      travelScore(Price12,Duration12,0,MinP,MaxP,MinD,MaxD,inf,Rank
            ),
62      Solution=sol(Price12, Duration12, 0, Rank, Flight1Info,
            Flight1ReturnInfo, _, _).
```

Per il calcolo dello score si utilizza il predicato *travelScore/9*. Questo predicato trova il rank di una soluzione come definito in 3.4.

```
63  wPrice(AP) :- AP is 1/3.
64  wDuration(AD) :- AD is 1/3.
65  wWaitingTime(AWT) :- AWT is 1/3.
66  travelScore(AvgPrice, AvgDuration, WaitingTime, MinP, MaxP, MinD,
        MaxD, CapWT, Rank) :-
67      NormalizedPrice is (AvgPrice - MinP) / (MaxP - MinP),
68      NormalizedDuration is (AvgDuration - MinD) / (MaxD - MinD),
69      NormalizedWaitingTime is (WaitingTime) / (CapWT),
70      wPrice(AP), wDuration(AD), wWaitingTime(AWT),
71      Rank is AP*NormalizedPrice + AD*NormalizedDuration + AWT*
            NormalizedWaitingTime.
```

dove:

- *wPrice* alla riga 61 è il peso da associare al prezzo nella funzione di valutazione,
- *wDuration* alla riga 62 è il peso da associare alla durata nella funzione di valutazione,
- *wWaitingTime* alla riga 63 è il peso da associare al tempo di attesa nella funzione di valutazione,
- *NormalizedPrice* alla riga 67 è il prezzo normalizzato,
- *NormalizedDuration* alla riga 68 è la durata di volo normalizzata,
- *NormalizedWaitingTime* alla riga 69 è l'attesa tra i due voli normalizzata, e
- *Rank* alla riga 71 è il rank calcolato usando i pesi e le misure normalizzate.

Attraverso questo codice Prolog appena descritto è possibile effettuare delle interrogazioni e ottenere, se esistono, le soluzioni. Un esempio di interrogazione è il seguente:

```
1  ?- fly2meet(cdg, lhr, bestsolution, no, 3600, yes, yes, Flights).
```

dove:

1. *"cdg"* è il codice IATA della prima città,
2. *"lhr* è il codice IATA della seconda città,
3. *"bestsolution"* è il tipo di ordinamento che vogliamo dare alle soluzioni,
4. *"no"* indica che vogliamo voli di sola andata,
5. *3600* è il massimo tempo di attesa che deve intercorrere tra l'arrivo dei due voli espresso in secondi,
6. *"yes"* indica che l'arrivo deve avvenire nello stesso aeroporto,
7. *"yes"* indica che non sono tollerati volitrai due aeroporti di partenza nella soluzioni, e
8. *Flights* è la variabile che memorizzerà una lista contenente le soluzioni.

4.1.2 Ricerca delle soluzioni

La ricerca delle soluzioni viene effettuata utilizzando il modulo *PrologMQI* appartenente alla libreria *swiplserver*. Dopo aver creato una istanza di PrologMQI e un thread Prolog al suo interno utilizzando il metodo *create_ thread* è possibile fare eseguire interrogazioni al thread utilizzando il metodo *query*. L'utilizzo del costrutto *with* crea un contesto di gestione delle risorse che permette di inizializzare le risorse all'inizio del blocco with e di rilasciare le risorse al termine del blocco.

```
1  with PrologMQI() as mqi:
2      with mqi.create_thread() as prolog_thread:
3          prolog_thread.query('consult(prologFacts/prolog_facts)')
4          prolog_thread.query('consult(fly2meet)')
5          for iata_code1 in first_city_iata_codes:
6              for iata_code2 in second_city_iata_codes:
7                  query = (f'fly2meet({iata_code1},'
8                           f'{iata_code2},'
9                           f'bestsolution,'
10                          f'{return_flag},'
11                          f'{waiting_time},'
12                          f'{same_airport_flag},'
13                          f'{new_city_flag},'
14                          f'Flights).')
15                 result = prolog_thread.query(query)
16                 if result:
17                     results.extend(result[0]['Flights'])
```

dove:

alla riga 3 , la prima interrogazione fa consultare il file *"prolog_facts.pl"* contenente i fatti Prolog che costituiranno la knowledge base,

alla riga 4 la seconda interrogazione consulta il file *"fly2meet.pl"* contenente i predicati necessari per effettuare le interrogazioni successive e effettuare ricerca della soluzione.

Data la coppia di codici IATA prima città, seconda città (o per ogni combinazione di codici IATA nel caso in cui le due città presentino più di un aeroporto), alla riga 17 viene eseguita la seguente interrogazione:

```
1 fly2meet(Airport1, Airport2, SortStrategy, Return, CapWT,
     SameAirport, NewCity, Flights)
```

Il predicato *fly2meet/8* permette di cercare le soluzioni all'interno della knowledge base utilizzando l'euristica definita in 3.4. Il predicato prende i seguenti argomenti:

1. *Airport1* è il codice IATA di un aeroporto appartenente alla prima città di partenza (e.g. cdg).
2. *Airport2* è il codice IATA di un aeroporto appartenente alla seconda città di partenza (e.g.lhr).
3. *SortStrategy* è la strategia di sorting da adottare per l'ordinamento delle soluzionio della interrogazione. I possibili valori che si possono dare a SortStrategy sono *"price"*, *"duration"*, *"waitingtime"* e *"bestsolution"*. L'ordinamento per best solution permette di ordinare le soluzioni utilizzando il rank.
4. *Return* ha valore di *"yes"* se è incluso il ritorno nelle soluzioni oppure *"no"* altrimenti.
5. *CapWT* indica il tempo di attesa massimo voluto dall'utente, altrimenti *"inf"*.
6. *SameAirport* vale *"yes"* se le due persone vogliono incontrarsi nello stesso aeroporto, *"no"* se basta loro incontrarsi nella stessa città e accettano di volare su aeroporti diversi.
7. *NewCity* vale *"yes"* se nelle soluzioni non sono ammessi voli diretti tra le due città, *"no"* altrimenti.
8. *Flights* è la lista contenente le soluzioni alla interrogazione Prolog.

Le soluzioni vengono restituite sotto forma di fatti Prolog nel formato descritto nella sezione 3.6.4 e contenute all'interno di *Flights*:

```
1 sol(AvgPrice, AvgDuration, WaitingTime, Rank, OutboundFlight1,
     ReturnFlight1, OutboundFlight2, ReturnFlight2).
```

Le soluzioni vengono restituite in formato JSON, in questo modo è possibile accedere e utilizzare facilmente le informazioni contenute in esso da Python e dal template HTML per la visualizzazione dei risultati da parte dell'utente.

4.1.3 Conversione delle offerte in fatti Prolog

La conversione delle offerte di volo trovate in fatti Prolog nel formato descritto in 3.6 avviene utilizzando un oggetto *FlightParser*. FlightParser permette di convertire i codici IATA e le offerte di volo JSON in fatti Prolog attraverso i metodi che mette a disposizione. I due principali metodi utilizzati sono *prolog_flight_parser* e *prolog_airport_parser*.

prolog_flight_parser prende per parametro un Array di offerte di volo in formato JSON e restituisce un Array di fatti Prolog nel formato *flight/9* se sono voli di sola andata oppure *itinerary/5* se sono coppie di voli di andata e voli di ritorno:

```
def prolog_flight_parser(self, all_flight_offers):
    prolog_facts = []
    for flight in all_flight_offers:
        itineraries = flight['itineraries']
        if len(itineraries) == 1:
            segment = itineraries[0]['segments'][0]
            origin, destination, carrier_code, flight_number,
               departure_date, arrival_date, duration = (
                self._get_segment_data(segment)
            )
            price = flight['price']['total']
            prolog_facts.append(
                f"flight({origin.lower()}, {destination.lower()},
                    {carrier_code}, {flight_number}, "
                f"{departure_date}, {arrival_date}, {duration}, {
                   price})."
            )
        else:
            segment1 = itineraries[0]['segments'][0]
            origin1, destination1, carrier_code1, flight_number1,
                departure_date1, arrival_date1, duration1 = (
                self._get_segment_data(segment1)
            )
            segment2 = itineraries[1]['segments'][0]
            origin2, destination2, carrier_code2, flight_number2,
                departure_date2, arrival_date2, duration2 = (
                self._get_segment_data(segment2)
            )
            price = flight['price']['total']
            prolog_facts.append(
                f"itinerary({origin1.lower()}, {destination1.
                   lower()},"
                f" f({carrier_code1}, {flight_number1}, {
                   departure_date1}, {arrival_date1}, {duration1
                   }),"
                f" f({carrier_code2}, {flight_number2}, {
                   departure_date2}, {arrival_date2}, {duration2
                   }),"
                f" {price})."
            )
```

```
31      return prolog_facts
```

prolog_airport_parser prende per parametro una insieme di codici IATA e restituisce un Array di fatti Prolog nel formato *airport/3*.

```
32  def prolog_airport_parser(self, all_airports):
33      prolog_facts = []
34      for airport in all_airports:
35          data = self.airports.airport_iata(airport)
36          country = pycountry.countries.get(name=data.country).
                alpha_2
37          iata_code = str(data.iata).lower()
38          city_name = data.city
39          country_code = country.lower()
40          prolog_facts.append("airport(" + iata_code + ", '" +
                city_name + "', " + country_code + ").")
41      return prolog_facts
```

I fatti Prolog così ottenuti possono essere scritti in un file *prolog_facts.pl* e successivamente essere consultati dal programma Prolog ed essere aggiunti alla propria knowledge base.

4.2 Frontend

4.2.1 Ricerca delle offerte di volo

In questa sezione viene descritto il processo attraverso la quale vengono cercate le informazioni riguardanti le offerte di volo, la descrizione dell'implementazione delle funzioni di utilità viene lasciata all'appendice (Sezione 4.2.3).

Le offerte di volo vengono memorizzate utilizzando tre strutture dati principali:

```
1  first_city_offers = []
2  second_city_offers = []
3  iata_codes = set()
```

dove:

first_city_offers è l'array contenente tutte le offerte di volo che partono dalla prima città,

second_city_offers è l'array contenente tutte le offerte di volo che partono dalla seconda città, e

iata_codes è l'insieme dei codici IATA utilizzati dalle offerte di volo. Queste tre strutture dati conterranno le informazioni che sarà necessario convertire in fatti Prolog.

La ricerca delle offerte di volo avviene in due casi differenti come illustrato nella sezione 3.3:

1. il caso in cui non siano stati specificati *target_countries* o *target_cities* nel quale si procede con una ricerca automatica delle destinazioni (Sezione 4.2.1), e

2. il caso in cui l'utente ha espresso almeno una preferenza su una città o un paese di destinazione, e in quel caso si procede con una ricerca delle offerte di volo sulle *target_cities* e *target_countries* (Sezione 4.2.1).

Ricerca della destinazione automatica

Nel caso in cui la ricerca delle destinazioni venga lasciata al sistema si procede cercando le ispirazioni di volo e successivamente si effettua una ricerca delle offerte di volo. Si procede nel seguente modo:

```
4 if target_cities is None and target_countries is None:
5     flight_inspirations = get_flight_inspirations(
          iata_departure_city_1, departure_date, max_base_price)
6     if flight_inspirations is None:
7         return render_template('display_results.html', error_msg=
              "No flight inspirations found.")
```

La riga 4 effettua un controllo sul campo *target_cities* e *target_countries* per controllare che l'utente non abbia inserito città o paesi di preferenza. La chiamata a *get_flight_inspirations* alla riga 5 avviene specificando il codice il codice IATA della prima di partenza, la data di partenza e il costo base massimo. Questa funzione a sua volta effettua una chiamata a *Flight Inspiration API* di Amadeus utilizzando questi parametri. Se *max_base_price* non viene specificato, di default vengono visualizzate tutte le offerte senza limitazione di prezzo. Le ispirazioni di volo trovate vengono memorizzate nella variabile *flight _inspirations* e utilizzate per la ricerca delle offerte di volo della prima città. A riga 7 se c'è stato un errore durante la ricerca delle ispirazioni di volo viene visualizzato un messaggio di errore nel template.

Dopo aver ottenuto le ispirazioni di volo, per ciascuna di esse si effettua una chiamata alla API *Flight Offer Search* di Amadeus.

```
8     first_city_destinations = set()
9     for flight in flight_inspirations:
10        fo = get_flight_offers_and_update_iata_codes(
              iata_departure_city_1, flight['destination'],
              departure_date, return_date, max_base_price,
              iata_codes)
11        if fo is not None:
12            first_city_offers.extend(fo)
13        if len(fo) > 0:
14            first_city_destinations.add(flight['destination'])
```

La chiamata della funzione *get_flight_offers_and_update_iata_codes* a riga 8 restituisce le offerte di volo e aggiorna l'insieme dei codici IATA da trasformare in fatti Prolog. Alla riga 12, se la ricerca per una determinata ispirazione di volo produce dei risultati, vengono aggiunti all'array *first_city _offers*. Alla riga 14, e l'array dei

risultati non è vuoto, viene aggiunto il codice IATA della destinazione all'insieme *first_ city _ destinations*.

La variabile *first_ city _ destinations* memorizza l'insieme delle destinazioni della prima città che verranno utilizzate successivamente per la ricerca delle offerte di volo per la seconda città. Successivamente viene effettuato un controllo su *new_ city*:

```
15      if not new_city:
16          fo = get_flight_offers_and_update_iata_codes(
                iata_departure_city_1, iata_departure_city_2,
                departure_date, return_date, max_base_price,
                iata_codes)
17          if fo is not None:
18              first_city_offers.extend(fo)
```

La riga 15 controlla il valore dell'opzione *new_ city*. Se questa opzione non è stata selezionata, vengono cercate anche le informazioni di voli diretti che partono dalla prima città e arrivano alla seconda città (riga 20).

Dopo aver fatto la ricerca delle offerte di volo per la prima città si cercano quelle per la seconda città di partenza.

```
19      for target_cities in first_city_destinations:
20          fo = get_flight_offers_and_update_iata_codes(
                iata_departure_city_2, target_cities, departure_date,
                return_date, max_base_price, iata_codes)
21          if fo is not None:
22              second_city_offers.extend(fo)
```

La ricerca avviene in maniera simile a quanto avviene per la prima città soltanto che invece di iterare su *flight _ inspirations* si itera su *first_ city _ destinations*: la riga 19 itera sull'insieme delle destinazioni appartenenti ai voli che hanno per origine la prima città, la riga 20 effettua la ricerca utilizzando su queste destinazioni utilizzando *get_ flight_ offers_ and_ update_ iata_ codes*. Se non è stata scelta l'opzione *new_ city* vengono cercate anche i voli diretti che partono dalla seconda città e arrivano alla prima città:

```
23      if not new_city:
24          fo = get_flight_offers_and_update_iata_codes(
                iata_departure_city_2, iata_departure_city_1,
                departure_date, return_date, max_base_price,
                iata_codes)
25          if fo is not None:
26              second_city_offers.extend(fo)
```

La riga 23 effettua un controllo sul *new_ city*, la riga 24 effettua la ricerca e la riga 25 aggiunge le offerte all'Array contenente le offerte di volo della seconda città.

Ricerca su target cities e target countries

Se l'utente ha inserito dei nomi di città o nomi di dei paesi èdi preferenza, la ricerca avviene iterando prima sui nomi delle città inserite e dopo sui nomi dei paesi. La ricerca sulle città inserite dall'utente avviene in questo modo:

```
27 if target_cities is not None:
28     destinations = target_cities.split(", ")
29     for dest in destinations:
30         iata_destination = get_iata_code(dest)
31         if iata_destination is None:
32             continue
33         response = get_flight_offers_and_update_iata_codes(
               iata_departure_city_1, iata_destination,
               departure_date, return_date, max_base_price,
               iata_codes)
34         if response is not None:
35             first_city_offers.extend(response)
36         response = get_flight_offers_and_update_iata_codes(
               iata_departure_city_2, iata_destination,
               departure_date, return_date, max_base_price,
               iata_codes)
37         if response is not None:
38             second_city_offers.extend(response)
```

Alla riga 27 viene controllato se l'utente ha scritto qualcosa nel form di ricerca, in tal caso si procede alla riga 28 le città separate da una virgola vengono memorizzate in un array *destinations*. Ciascun nome di città viene convertito in un codice IATA utilizzando la funzione *get_iata_code* alla riga 30.

Alla riga 32, se il nome della città non corrisponde a nessun codice IATA nel presente nel Database di Amadeus allora si procede con l'iterazione succesiva.

Alla riga 33 e 36, se il nome viene risolto in un codice IATA, si effettua una ricerca utilizzando la funzione *get_flight_offers_and_update_iata_codes* per le due le città di partenza.

La ricerca sui paesi inseriti dall'utente nel form di ricerca avviene in questo modo:

```
39 if target_countries is not None:
40     countries = target_countries.split(", ")
41     for dest in countries:
42         target_country_airports_iata =
               get_airports_from_country_name(dest)
43         for iata in target_country_airports_iata:
44             response = get_flight_offers_and_update_iata_codes(
                   iata_departure_city_1, iata, departure_date,
                   return_date, max_base_price, iata_codes)
45             if response is not None:
46                 first_city_offers.extend(response)
47             response = get_flight_offers_and_update_iata_codes(
                   iata_departure_city_2, iata, departure_date,
                   return_date, max_base_price, iata_codes)
48             if response is not None:
49                 second_city_offers.extend(response)
```

Alla riga 39 viene effettuato un controllo sulla variabile *target_countries* per controllare se l'utente ha inserito qualcosa nel form di ricerca, in caso affermativo vengono ottenuti i nomi dei paesi e vengono memorizzati nella variabile *countries*.

Alla riga 41 inizia l'iterazione sui nomi dei paesi contenuti in *countries*.

Alla riga 42 la chiamata alla funzione *get_airports_from_country _name* restituisce i codici IATA dei maggiori aeroporti di un determinato paese. Successivamente, per ciascuno dei codici IATA ottenuto viene fatta la ricerca per le due città di partenza (riga 44 e 47) e vengono estesi i due array contenenti le offerte di volo (riga 46 e 49).

4.2.2 Struttura del server

Fly2Meet consiste in un server programmato usando Python e il framework Flask. Il server è organizzato nel seguente modo:

```
1  app = Flask(__name__, static_folder='static', template_folder='
       templates')
2
3  @app.route('/search_flights', methods=['POST'])
4  def search_flights():
5      # corpo della funzione search_flights
6      return render_template("display_results.html", prolog_results
           =results, include_return=include_return)
7
8  @app.route('/')
9  def index():
10     return render_template('index.html')
11
12
13 if __name__ == '__main__':
14     app.run()
```

Flask permette di gestire il routing. La gestione del routing è una funzione di Flask importante che permette di associare delle URL specifiche a delle funzioni che definiscono il modo in cui il Server deve rispondere a determinate richieste da parte dei client. Questa cosa viene fatta utilizzando dei *decoratori* da anteporre alla definizione della funzione. Dato che il decoratore alla riga 8 si trova sopra la definizione della funzione *index* alla riga 9, il server quando un utente accede alla URL della pagina principale di Fly2Meet (avente URL *'/'*) chiamerà la funzione decorata. La chiamata di questa funzione esegue la chiamata a *render_template* alla riga 10. Questa istruzione fa le seguenti azioni:

1. *carica il template HTML* avente nome *'index.html'* situato nella cartella predefinita definita da Flask (in questo caso la cartella *'templates'*).

2. *renderizza il template HTML.*

3. *restituisce la pagina HTML index.html* come risposta HTTP al browser dell'utente.

All'interno del body della pagina HTML *index.hml* si trova un form da riempire per effettuare la ricerca:

```
1  <form method="POST" action="{{ url_for('search_flights') }}">
2      <label for="departure_city_1">Departure City 1:</label>
3      <input type="text" id="departure_city_1" name="
           departure_city_1" placeholder="e.g. Paris" autocomplete="
           off" required>
4      <label for="departure_city_2">Departure City 2:</label>
5      <input type="text" id="departure_city_2" name="
           departure_city_2" placeholder="e.g. London" autocomplete="
           off" required>
6
7      <label for="departure_date">Departure Date:</label>
8      <input type="date" id="departure_date" name="departure_date"
           required>
9
10     <label for="return_date">Return Date (optional):</label>
11     <input type="date" id="return_date" name="return_date">
12
13     <!-- altri filtri di ricerca avanzati...-->
14
15     <div class="checkbox">
16         <input type="checkbox" id="show_advanced_options" name="
               show_advanced_options">
17         <label for="show_advanced_options">Show Advanced Options<
               /label>
18     </div>
19
20     <button type="submit">Search</button>
21 </form>
```

Gli attributi *method* e *action* alla riga 1 specificano come il modulo deve comportarsi:

- il valore di *method="POST"* indica che i dati del modulo saranno inviati al server Web utilizzando il metodo HTTP *POST*.
- il valore di *action = {{ url_for('search _flights') }}* specifica l'URL del server a cui verranno inviati i dati del modulo quando l'utente preme il pulsante *"Search"*.

Alla pressione del tasto *"Search"*, dato che nel server la funzione *search_flights* è stata decorata con

```
1 @app.route('/search_flights', methods=['POST'])
```

verrà chiamata quella funzione. Il primo campo del decoratore indica la route, mentre *methods=['POST']* in questo caso indica che la route accetta soltanto richieste HTTP di tipo POST.

La funzione *search_flights* effettua una ricerca delle offerte di volo partendo dai dati inseriti nel form. Per fare ciò è necessario prima accedere ai dati inviati dal client attraverso il form che ha compilato.

Per fare ciò si utilizza il metodo *request.form.get* che permette di recuperare i valori dei campi di un form inviato al server attraverso una richiesta POST. L'accesso

avviene dando come argomento a request.form.get il valore dell'attributo "name" di ciascun input. Per ciascun filtro di ricerca opzionale viene fatto un controllo sul valore. Nel caso in cui un campo di input non sia stato compilato oppure non sia stata selezionata allora gli viene assegnato un valore di default.

```
@app.route('/search_flights', methods=['POST'])
def search_flights():
    departure_city_1 = request.form.get('departure_city_1')
    departure_city_2 = request.form.get('departure_city_2')
    departure_date = request.form.get('departure_date')
    return_date = request.form.get('return_date') if request.form
       .get('return_date') != '' else None
    max_base_price = request.form.get('max_base_price') if
       request.form.get('max_base_price') != '' else None
    max_duration = request.form.get('max_duration') if request.
       form.get('max_duration') != '' else None
    max_wait_time = int(request.form.get('max_wait_time'))*60 if
       request.form.get('max_wait_time') != '' else None
    target_cities = request.form.get('target_cities') if request.
       form.get('target_cities') != '' else None
    target_countries = request.form.get('target_countries') if
       request.form.get('target_countries') != '' else None
    same_airport = False if request.form.get('same_airport') is
       None else True
    new_city = False if request.form.get('new_city') is None else
        True
```

Dai nomi delle città inseriti viene cercato il corrispettivo codice IATA attraverso una chiamata a *Aiport & City Search* di Amadeus API:

```
    iata_departure_city_1 = get_iata_code(departure_city_1)
    if iata_departure_city_1 is None:
        return render_template('display_results.html', error_msg=
           "First departure city not found.")
    time.sleep(1)

    iata_departure_city_2 = get_iata_code(departure_city_2)
    if iata_departure_city_2 is None:
        return render_template('display_results.html', error_msg=
           "Second departure city not found.")
```

la chiamata alla funzione *get_iata_code* alla riga 14 e 19 restituisce il codice IATA rispettivamente della prima e della seconda città di partenza. Se non esiste alcun codice IATA nel database di Amadeus corrispondente a uno dei due nomi di città inseriti viene renderizzato il template *'display_results* stampando il messaggio di errore "First city not found." o "Second city not found." a seconda del caso (riga 16 e 21).

4.2.3 Funzioni di utilità

Definire delle funzioni di utilità per il server è utile perché permette il riutilizzo del codice, aggiunge uno strato di astrazione maggiore alle API nascondendo dettagli

tecnici durante la chiamata, rende la manutenzione del sistema più semplice.

Le chiamate alle diverse API di Amadeus vengono effettuate utilizzando un oggetto *APISession*.

```
api_session = APISession(api_key, api_secret)
```

APISession è supportata da un oggetto di tipo Session che permette di mantenere lo stato tra le molteplici richieste HTTP effettuate alle API di Amadeus. APISession prende per parametri nel costruttore *api_key* e *api_secret* e le utilizza per ottenere l'access Token.

L'access Token ha una durata massima di tempo dopo la quale necessita di essere rinnovato. APISession provvede a rinnovare automaticamente l'access Token allo scadere del tempo alla prima richiesta effettuata dopo che il vecchio access Token non è più utilizzabile. Utilizzando *api_session* per effettuare le richieste HTTP alla API Amadeus vengono realizzate le seguenti funzioni di utilità.

La funzione *get_iata_code* è una funzione che effettua una chiamata a *Airport & City Search API* di Amadeus. Prende per parametro il nome della città e restituisce il codice IATA della città se è presente nel database di Amadeus, None altrimenti.

```
def get_iata_code(name):
    url = "https://test.api.amadeus.com/v1/reference-data/
        locations"
    params = {
        "subType": "CITY",
        "keyword": name,
        "view": "LIGHT"
    }
    response = api_session.get(url, params=params)
    if response is None:
        print("Error requesting IATA")
        return None
    else:
        data = response.json()
        if data['meta']['count'] == 0:
            print(f"No IATA code found for {name}")
            return None
        else:
            return data['data'][0]['iataCode']
```

La funzione *get_flight_inspirations* effettua una chiamata a *Flight Inspiration Search*. La funzione prende per parametri il codice IATA da cui si desidera partire, la data di partenza e il costo base massimo che si desidera pagare (se non specificato il valore deve essere None). Se la chiamata va a buon fine di ritorno restituisce le ispirazioni di volo per una determinata città.

```
def get_flight_inspirations(_origin, _departure_date, _max_price)
    :
    url = 'https://test.api.amadeus.com/v1/shopping/flight-
        destinations'

    params = {
```

```
        'origin': _origin,
        'departureDate': _departure_date,
        'nonStop': 'true',
    }
    if _max_price is not None:
        params['maxPrice'] = _max_price
    response = api_session.get(url, params=params)
    if response is not None:
        data = response.json()
        return data['data']
    else:
        print("Error requesting Flight Inspirations")
        return None
```

La funzione *get_flight_offers* permette di effettuare una chiamata a *Flight Offer Search* di Amadeus. La funzione prende per parametri il codice IATA della città di partenza, il codice IATA della città di destinazione, la data di partenza e (opzionale) la data di ritorno e il costo base massimo desiderato. Se non specificato il valore di *_return_date* o *_max_base_price* deve essere None. Se la chiamata alla API va a buon fine le offerte di volo trovate vengono restituite in formato JSON, altrimenti viene restituito None.

```
def get_flight_offers(_origin, _destination, _departure_date,
    _return_date, _max_base_price):
    url = "https://test.api.amadeus.com/v2/shopping/flight-offers
        "
    params = {
        "originLocationCode": _origin,
        "destinationLocationCode": _destination,
        "departureDate": _departure_date,
        "currencyCode": "EUR",
        "nonStop": "true",
        "adults": 1
    }
    if _return_date is not None:
        params["returnDate"] = _return_date
    if _max_base_price is not None:
        params["maxPrice"] = _max_base_price

    response = api_session.get(url, params=params)
    if response is None:
        print("Error requesting Flight Offers")
        return None
    else:
        data = response.json()
        return data
```

Un'altra funzione di utilità è che fa utilizzo di *get_flight_offers* è la funzione *get_flight_offers _and _update_iata_codes* che aggiorna i codici IATA contenuti all'interno del set *iata_codes* e restituisce le offerte di volo trovate.

```
def get_flight_offers_and_update_iata_codes(origin, destination,
    departure_date, return_date, max_base_price, iata_codes):
    fo = get_flight_offers(origin, destination, departure_date,
        return_date, max_base_price)
    if fo is None:
        return None
    else:
        for tmp in fo['data']:
            for itinerary in tmp['itineraries']:
                for segment in itinerary['segments']:
                    iata_codes.add(segment['departure']['iataCode
                        '])
                    iata_codes.add(segment['arrival']['iataCode'
                        ])
        return fo['data']
```

La funzione *get_airports_from_country_name* permette di trovare i codici IATA di un determinato paese partendo dal suo nome. Per fare ciò utilizziamo la libreria *pycountry*[2] che provvede un database contenente le informazioni in standard ISO riguardanti i paesi. L'istruzione *pycountry.countries.get* utilizza il nome del paese come filtro di ricerca per ottenere le informazioni riguardanti un determinato paese, tra cui l'identificatore in formato ISO del paese. Utilizzando il codice ISO si può effettuare una ricerca all'interno del file *"airport-codes.csv"* che contiene informazioni su tutti gli aeroporti in modo strutturato e si collezionano tutti gli aeroporti del paese preso per parametro.

```
def get_airports_from_country_name(country_name):
    country_data = pycountry.countries.get(name=country_name)
    if country_data:
        filename = "utils/airport-codes.csv"
        with open(filename, newline="", encoding="utf-8") as
            csvfile:
            reader = csv.reader(csvfile)
            next(reader, None)
            iata_codes = set()
            for row in reader:
                atype = row[1]
                iso_country = row[5]
                iata_code = row[9]
                if iata_code != "" and iso_country ==
                    country_data.alpha_2 and atype == "
                    large_airport":
                    iata_codes.add(iata_code)
            return iata_codes
```

[2]https://pypi.org/project/pycountry/

Capitolo 5

Testing

In questo capitolo vengono descritti brevemente gli strumenti utilizzati per effettuare il testing della applicazione Web, le modalità con cui sono stati effettuati i test e la progettazione dei casi di test e i risultati. Questo capitolo è diviso in due parti: la prima parte mostra come è stato effettuato il testing dell'applicazione della parte del servizio sviluppata utilizzando Python (Sezione 5.1), mentre la seconda parte mostra come è stato effettuato il testing del programma Prolog utilizzato (Sezione 5.2).

5.1 Testing Python

pytest[1] è un framework che permette di scrivere tests in poche righe di codice e leggibili e permette di fare test di applicazioni Python più o meno complesse. Per testare Fly2Meet è necessario impostare il flag di testing dell'applicazione Flask su "True".

```
app.testing = True
```

Quando il flag di testing è attivato, Flask apporta alcune modifiche al comportamento dell'applicazione per semplificare il testing unitario e l'integrazione. Ad esempio le eccezioni non vengono propagate per permettere ai tests di rilevarle più facilmente. Attivare la modalità di testing semplifica il processo di verifica del comportamento dell'applicazione senza influire sul comportamento di produzione. Per la scrittura dei tests viene seguita la convenzione di pytest secondo la quale i file devono avere il prefisso *test_ *.py* e i metodi al suo interno devono avere il prefisso *test_* per essere trovati da pytest. Un esempio di test contenuto all'interno di *test_ app.py* è il seguente:

```
def test_index():
    with app.test_client() as client:
        response = client.get('/')
        assert response.status_code == 200
        assert b'<!DOCTYPE html>' in response.data
```

[1] https://docs.pytest.org/en/7.4.x/

In questo test:

alla riga 2 viene creato un client di test per l'applicazione Flask. Il with statement garantisce che il client venga chiuso correttamente alla fine del test. Il client di test simula le richieste HTTP all'applicazione come se fosse un client reale,

alla riga 3 Il test esegue una richiesta HTTP GET alla radice *"/"* dell'applicazione utilizzando il client di test e memorizza la risposta ottenuta nella variabile *response*,

alla riga 4 viene utilizzato *assert* per controllare che il codice di stato della risposta HTTP sia 200, che indica che la risposta HTTP sia stata elaborata con successo, e

alla riga 5 viene verificato se il contenuto della risposta HTTP contiene la stringa *"<!DOCTYPE html>"* per assicurarsi che la risposta contenga del codice HTML valido.

Attraverso il metodo *client.post* è possibile effettuare una richiesta HTTP POST alla route *'/search_flights* dell'applicazione utilizzando il client di test e controllare la risposta attraverso una variabile response.

```
7  def test_search_flights_simple():
8      with app.test_client() as client:
9          response = client.post('/search_flights', data={
10             'departure_city_1': 'Paris',
11             'departure_city_2': 'London',
12             'departure_date': '2023-08-14',
13             'return_date': '',
14             'max_base_price': '',
15             'max_duration': '',
16             'max_wait_time': '',
17             'target_cities': '',
18             'target_countries': '',
19             'different_city': 'on',
20         })
21         assert response.status_code == 200
22         assert b'<!DOCTYPE html>' in response.data
```

Questo test:

alla riga 7 crea un client di test

alla riga 8 effettua una richiesta HTTP POST alla route *"/search_flights"* utilizzando il client di test,

alla riga 20 controlla che il codice di stato della risposta HTTP sia 200, ossia che la richiesta HTTP sia stata elaborata con successo, e

alla riga 21 controlla che la risposta HTTP contenga la stringa *"<!DOCTYPE html>"* che indica che la risposta contiene del codice HTML valido.

I test sono stati progettati per controllare che vengano gestiti correttamente i casi in cui c'è errore. Ad esempio questo test controlla che venga nel caso in cui l'utente inserisca una città inesistente gli venga mostrato un messaggio di errore.

```
23 def test_search_flights_invalid_city_1():
24     with app.test_client() as client:
25         response = client.post('/search_flights', data={
26             'departure_city_1': 'NonExistingCity1',
27             'departure_city_2': 'London',
28             'departure_date': '2023-08-14',
29             'return_date': '2023-08-20',
30             'max_base_price': '500',
31             'max_duration': '6',
32             'max_wait_time': '3',
33             'target_cities': 'Madrid, Barcelona',
34             'target_countries': 'Italy, France',
35             'different_city': 'on',
36         })
37         assert response.status_code == 200
38         assert b'First departure city not found.' in response.
               data
```

Questo test:

alla riga 24 effettua una richiesta HTTP POST alla route *"/search_flights"* utilizzando il client di test,

alla riga 36 controlla che il codice di stato della risposta HTTP sia 200, ossia che la richiesta HTTP sia stata elaborata con successo, e

alla riga 37 controlla che la risposta HTTP contenga la sotto-stringa *"First departure city not found."*, che è ciò che ci aspettiamo.

5.2 Testing Prolog

Il testing del codice Prolog è stato effettuato utilizzando il framework *Prolog Unit Tests* sviluppato da SWI Prolog. Questo framework permette di testare Prolog utilizzando delle *Unit Test Box* racchiuse tra i due predicati *begin_tests/1* e *end_tests/1* e permettono di automatizzare il testing del codice Prolog.

```
1  :- begin_tests(fly2meet).
2
3  :- consult(fly2meet).
4
5  :- dynamic(airport/3).
6  :- dynamic(flight/8).
7  :- dynamic(itinerary/5).
8
9  setup1 :-
10     consult('prologFacts/test1.pl').
11
```

```
12 setup2 :-
13     consult('prologFacts/test2.pl').
14
15 teardown1 :-
16     retractall(airport(_, _, _)),
17     retractall(flight(_, _, _, _, _, _, _, _)).
18
19 teardown2 :-
20     retractall(airport(_, _, _)),
21     retractall(itinerary(_, _, _, _, _)).
22
23 % Unit Tests Boxes
24
25 :- end_tests(fly2meet).
```

All'inizio di ciascun test per creare l'ambiente di esecuzione necessario vengono utilizzate le due procedure di setup *setup1* e *setup2*. La procedura *setup1* permette di preparare l'ambiente consultando il file *test1.pl* contenente i fatti *flight/8* e *airport/3*. La procedura *setup2*, invece, permette di preparare l'ambiente consultando il file *test2.pl* contenente i fatti *itinerary/5* e *airport/3*. Avere due procedure distinte di setup dell'ambiente di esecuzione dei test è stata una scelta necessaria data la distinzione che c'è tra la rappresentazione delle offerte di volo in *flight/8* e *itinerary/5*.

Al termine dell'esecuzione di ciascuna Unit Test Box vengono utilizzate *teardown1* e *teardown2* per riportare l'ambiente di esecuzione a prima dell'esecuzione. In questo caso vengono rimossi dalla knowledge base tutti i *airport/3* o *itinerary/5* in esso presenti, ottenuti dalla consultazione dei file nella fase di setup. Anche in questo caso l'utilizzo di due funzioni di cleanup è dovuto alla diversa rappresentazione in *flight/8* e *itinerary/5*. Un esempio di Prolog Unit Test Box è il seguente:

```
21 test(fly2meet_case1, [setup(setup1), cleanup(teardown1)]) :-
22     once(fly2meet(cdg, lhr, bestsolution, no, 4000, yes, yes,
           Flights)),
23     assertion(is_list(Flights)),
24     sort(4, @=<, Flights, SortedFlights),
25     assertion(Flights = SortedFlights).
```

dove:

fly2meet_case1 alla riga 21 è il nome del test,

setup(setup1) alla riga 21 specifica la procedura di configurazione che deve essere eseguita prima dell'esecuzione del test, in questo caso "setup1",

cleaup(teardown1) alla riga 21 specifica la procedura di ripristino che deve essere eseguita dopo l'esecuzione del test, in questo caso "teardown1",

once(fly2meet(cdg, lhr, bestsolution, no, 4000, yes, yes, Flights)) alla riga 22 effettua una interrogazione alla base di conoscenza utilizzando il predicato *fly2meet/8*,

assertion(is_list(Flights)) alla riga 23 verifica che la variabile *Flights* sia una effettivamente una lista,

sort(4, @=<, Flights, SortedFlights) alla riga 24 ordina la lista contenente le soluzioni utilizzando in modo crescente e la memorizza all'interno della variabile *SortedFlights*, e

assertion(Flights = SortedFlights) alla riga 25 verifica se la lista originale è uguale alla lista dopo l'ordinamento, così facendo verifica che la lista sia ordinata sul quarto parametro.

Utilizzando diverse combinazioni di parametri da utilizzare con *fly2meet/8* è possibile ottenere un'ampia gamma di test differenti, con la quale è possibile coprire tutti i possibili casi di utilizzo reali del sistema Fly2Meet.

Capitolo 6

Risultati

In questo capitolo viene mostrato brevemente ciò che è stato realizzato attraverso il prototipo richiesto dalla tesi.

6.1 Interfaccia grafica

Dal progetto di interfaccia descritto nella sezione 3.5 è stata prodotta l'interfaccia utente finale. Come descritto nella fase di progettazione, quando l'utente accede al servizio si fronte a sé l'interfaccia con il form di ricerca *minimale* di Fly2Meet per effettuare ricerche veloci, nascondendo i filtri di ricerca avanzati (Figura 6.1).

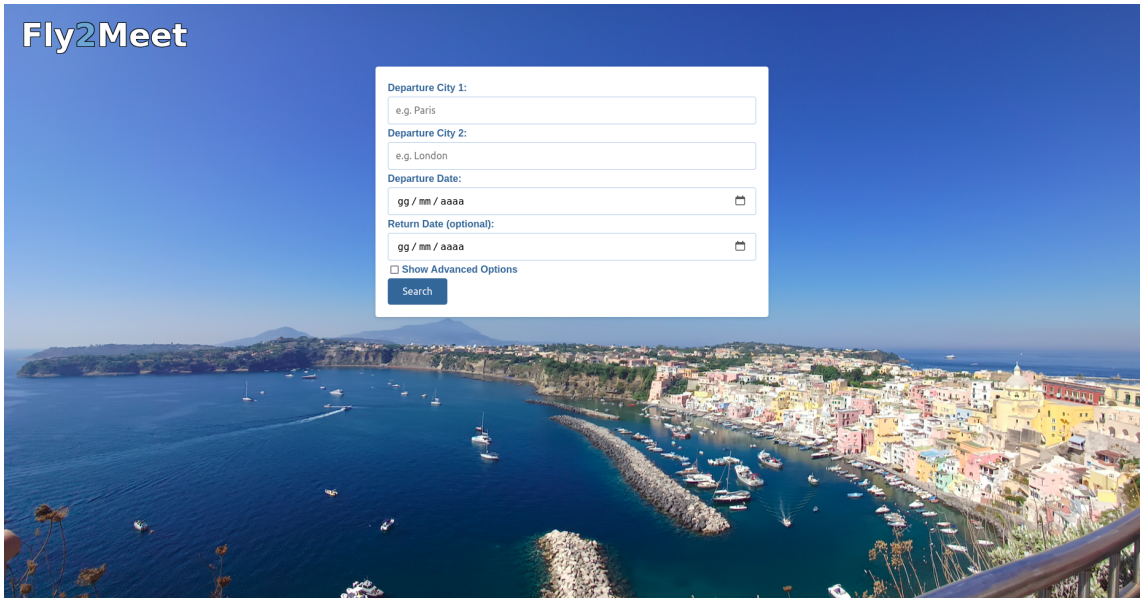


Figura 6.1: form di ricerca minimale utilizzato dal prototipo di Fly2Meet.

La checkbox *"Show Advanced Options"* permette di mostrare e nascondere su richiesta dell'utente i filtri di ricerca avanzati per personalizzare la ricerca in base alle loro preferenze, eliminando i risultati non pertinenti e mostrando solo ciò che interessa all'utente (Figura 6.2).

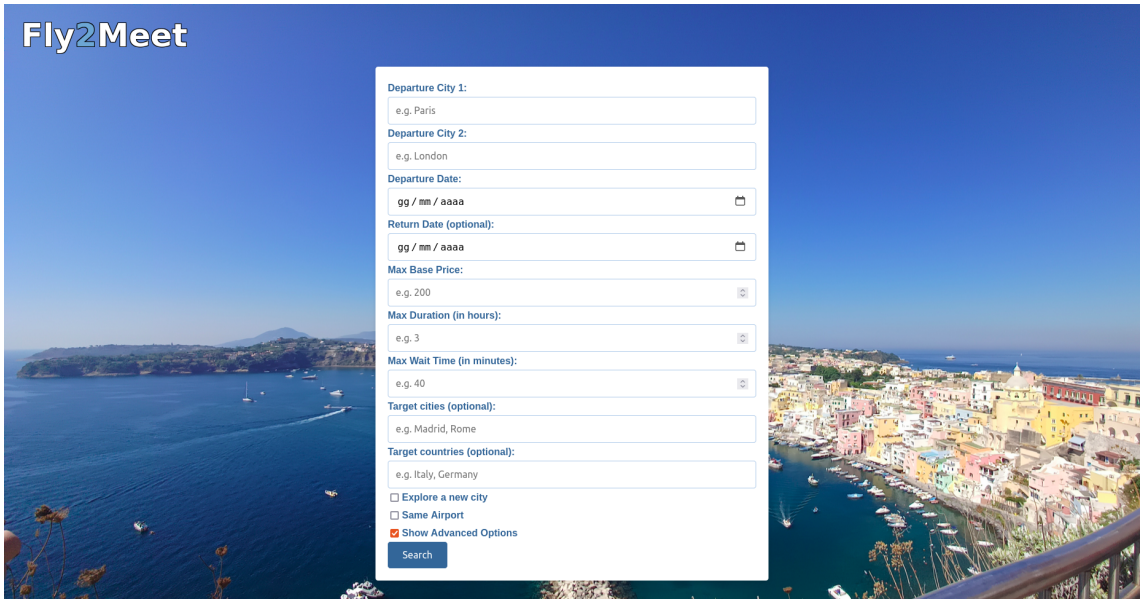


Figura 6.2: form di ricerca avanzato utilizzato dal prototipo di Fly2Meet.

Attraverso il form di ricerca avanzato è possibile effettuare ricerche più precise (Figura 6.3).

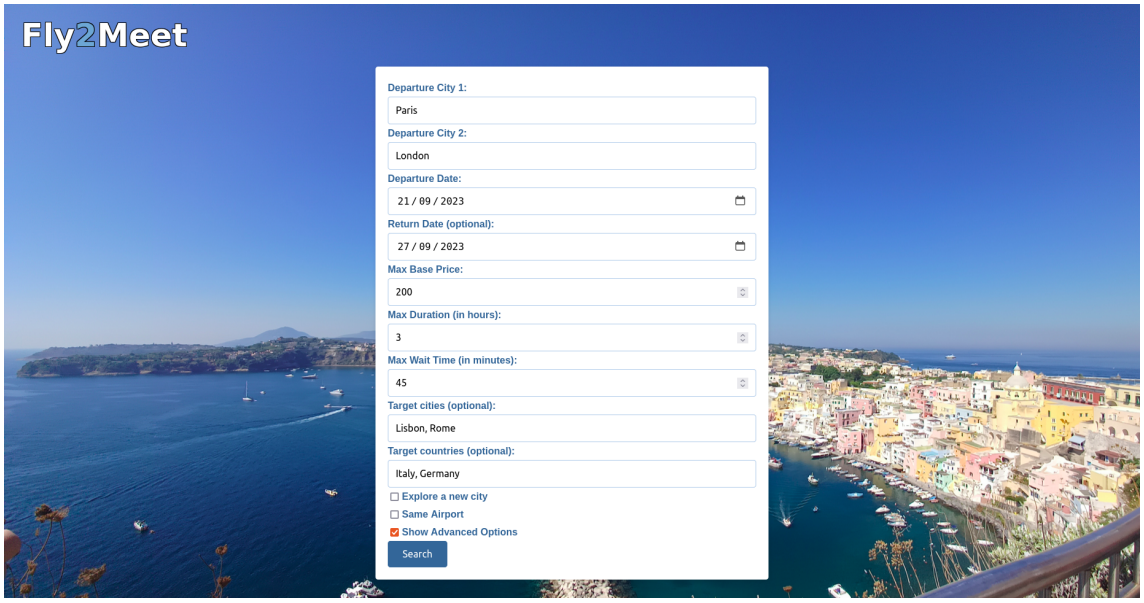


Figura 6.3: esempio di ricerca avanzata

6.2 Esempi di ricerca

Per eseguire una ricerca semplice è necessario compilare *"Departure City 1"*, che è il nome della prima città di partenza, *"Departure City 2"*, che è il nome della seconda città e *"Departure Date"*, che è il nome della data di partenza (Figura 6.4).

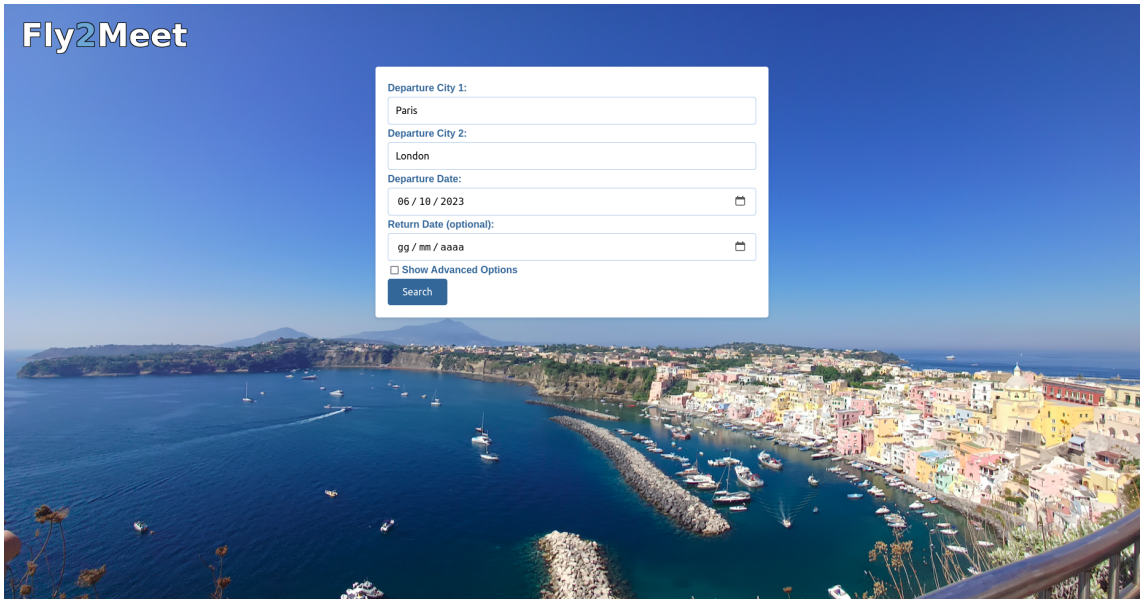


Figura 6.4: form di ricerca compilato per trovare il punto di incontro tra una persona di Parigi e una di Londra

.

I risultati vengono restituiti come coppie di offerte di volo contenenti le informazioni dell'offerta di volo come carrier code, carrier number, orari di partenza e arrivo, durate di volo e costo delle due offerte di volo (Figura 6.5).

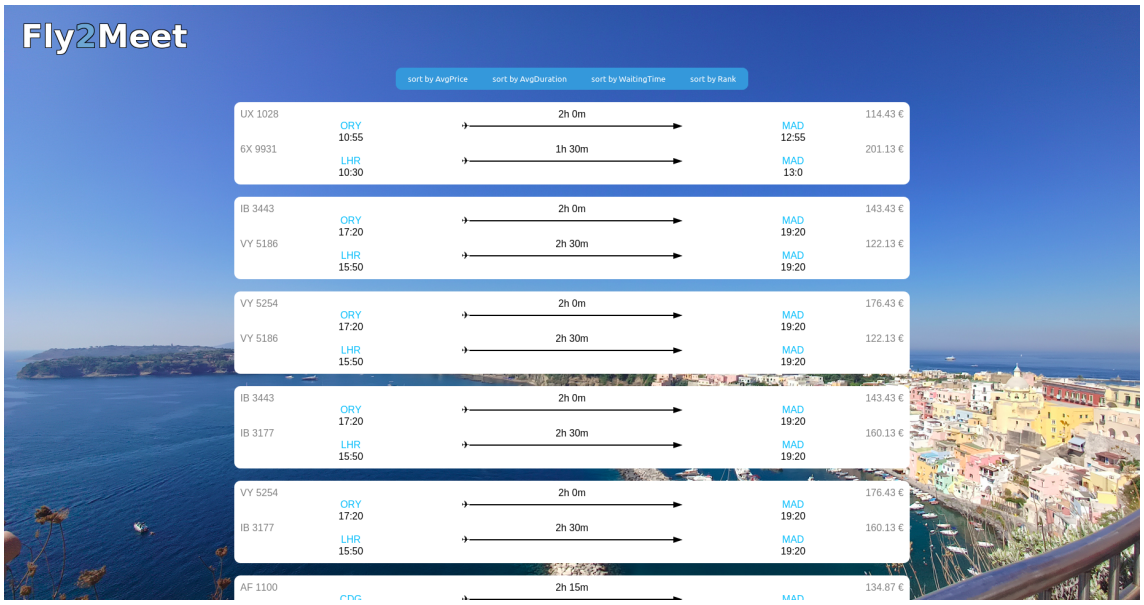


Figura 6.5: Risultati della ricerca dei voli utilizzando solo la data di ritorno.

Se l'utente ha compilato anche il campo *"Return Date"* vengono mostrate le soluzioni di volo che includono anche un volo di ritono, con annesse informazioni (Figura 6.6).

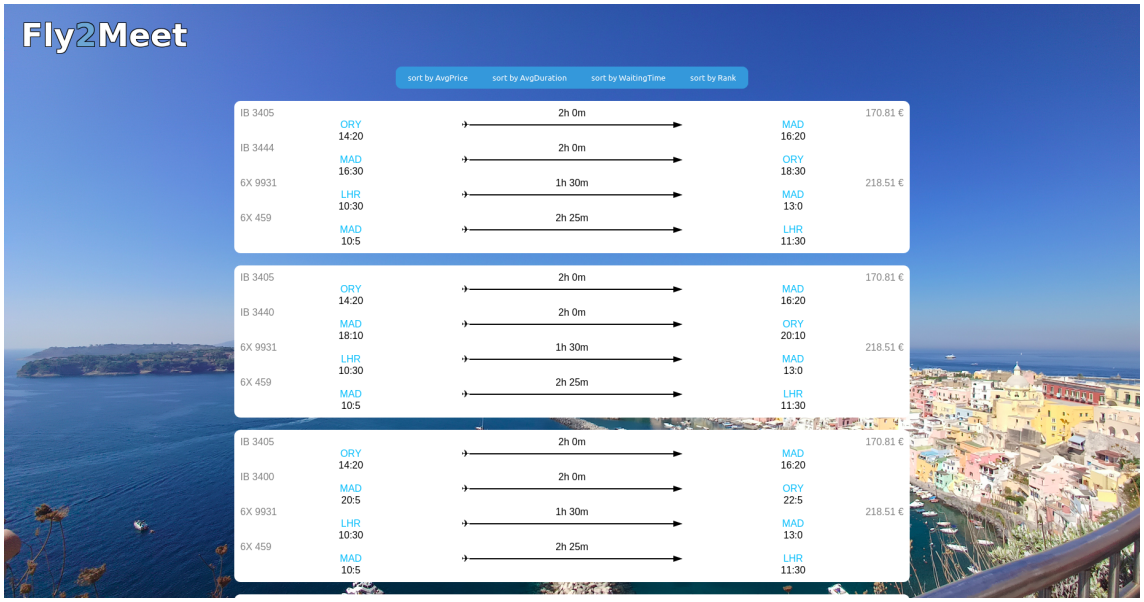


Figura 6.6: Risultato di una ricerca dove è stata scelta anche una data di ritorno.

Capitolo 7

Conclusioni e sviluppi futuri

7.1 Sommario della tesi

Lo scopo di questa tesi sono stati la progettazione e la realizzazione del prototipo di un servizio Web utile a organizzare viaggi aerei per fare incontrare due persone provenienti da parti diverse del mondo, riducendo costi, durate complessiva dei voli e tempi di attesa. Il prototipo è stato realizzato utilizzando il linguaggio di programmazione Prolog, il framework Python per lo sviluppo di servizi RESTful, e le API del servizio Web Amadeus per l'identificazione dei voli.

Dopo aver brevemente illustrato il retroterra tecnologico necessario per l'implementazione del progetto (Capitolo 2), sono state discussi i requisiti e la progettazione (Capitolo 3) e l'implementazione (Capitolo 4) del sistema Fly2Meet, aver mostrato come è stato effettuato il testing (Capitolo 5) del prototipo e aver mostrato alcune funzionalità che sono state realizzate dal prototipo (Capitolo 6), passiamo a un riepilogo dei risultati raggiunti con questa tesi.

Adesso possiamo concludere la tesi con una valutazione critica del lavoro svolto (Sezione 7.2) e una previsione degli sviluppi futuri che avrà il prodotto software sviluppato (Sezione 7.3).

7.2 Valutazione critica del lavoro svolto

Il prototipo realizzato raggiunge gli obiettivi fissati dalla tesi: con questo prototipo è possibile trovare soluzioni di viaggio convenienti che permettano a due persone provenienti da diverse parti del mondo di incontrarsi in maniera efficiente, bilanciando i costi, le durate dei viaggi e i tempi di attesa, il tutto attraverso una interfaccia grafica che permette l'interazione con l'utente. Attraverso questo prototipo due persone possono automatizzare il processo di ricerca e pianificazione e consultare un gran numero di offerte di volo.

Ciò nonostante, il prototipo realizzato presenta ancora delle limitazioni che, se superate possono raggiungere gli obiettivi della tesi in maniera più efficace. Possibili miglioramenti che possono essere apportati al sistema sono il calcolo in background e

la parallelizzazione del calcolo dei risultati in modo da restituire i risultati all'utente mano a mano che vengono calcolati, estendere la ricerca a voli con più scali, estendere il servizio a un numero di persone superiore a due, permettere la prenotazione dei voli direttamente dal servizio.

L'impatto ambientale sta diventando sempre più importante. Calcolare le emissioni di carbonio di ciascun volo e cercare i voli più efficienti dal punto di vista energetico può aiutare a mitigare l'impatto ambientale dei voli aerei.

Il testing è stato effettuato sul prototipo per assicurarsi il corretto funzionamento del sistema. Per vincoli di tempo e mancanza di accesso ai potenziali utenti finali, purtroppo non è stato possibile avere un feedback diretto dagli utenti per ottenere informazioni su come il prodotto viene percepito e utilizzato nel mondo reale.

7.3 Sviluppi futuri

Sviluppi futuri del sistema Fly2Meet, anche alla luce delle limitazioni identificate sopra, comprendono:

Efficientamento del sistema di ricerca attraverso una parallelizzazione del calcolo delle soluzioni con un conseguente miglioramento in termini di prestazioni, scalabilità e reattività del sistema,

Generalizzazione a più persone e più scali per rendere il servizio più flessibile alle esigenze degli utenti, permettendo a più persone di incontrarsi e ammettendo anche voli con più scali tra le soluzioni possibili se l'utente lo desidera,

Ricerca delle soluzioni con minore impatto ambientale così da permettere alle compagnie aeree di monitorare e ridurre le emissioni di CO2 associate ai viaggi aerei, nonché privilegiare soluzioni con minore impatto ambientale,

Permettere la prenotazione dei voli attraverso il sistema di Fly2Meet così da semplificare ulteriormente il processo di pianificazione dei viaggi aerei e migliorare la user experience, e

Valutazione con potenziali utenti finali per avere un feedback diretto dall'utente, valutare l'usabilità del sistema e eventualmente migliorare delle decisioni di progettazione.

Capitolo 8

Codice Prolog

```
wPrice(AP) :- AP is 1/3.
wDuration(AD) :- AD is 1/3.
wWaitingTime(AWT) :- AWT is 1/3.

fly2meet(Airport1, Airport2, SortStrategy, Return, CapWT,
   SameAirport, yes, Flights) :-
    airport(Airport1, _, _), airport(Airport2, _, _), dif(
       Airport1, Airport2),
    minAndMax(price, Return, MaxP, MinP),
    minAndMax(duration, Return, MaxD, MinD),
    findall(FCombo, eligibleFlights(Airport1, Airport2, MaxP,
       MinP, MaxD, MinD, Return, CapWT, SameAirport, FCombo),
       TmpFlights),
    sortByStrategy(TmpFlights, SortStrategy, Flights).

fly2meet(Airport1, Airport2, SortStrategy, Return, CapWT,
   SameAirport, no, Flights) :-
    airport(Airport1, _, _), airport(Airport2, _, _), dif(
       Airport1, Airport2),
    minAndMax(price, Return, MaxP, MinP),
    minAndMax(duration, Return, MaxD, MinD),
    findall(FCombo, eligibleFlights(Airport1, Airport2, MaxP,
       MinP, MaxD, MinD, Return, CapWT, SameAirport, FCombo),
       TmpFlights1),
    findall(FCombo, directFlight(Airport1, Airport2, MaxP, MinP,
       MaxD, MinD, Return, FCombo), TmpFlights2),
    findall(FCombo, directFlight(Airport2, Airport1, MaxP, MinP,
       MaxD, MinD, Return, FCombo), TmpFlights3),
    append([TmpFlights1,TmpFlights2,TmpFlights3], TmpFlights),
    sortByStrategy(TmpFlights, SortStrategy, Flights).


solutionAndRank(Airport1, Airport2, Airport3, Airport4, MaxP,
   MinP, MaxD, MinD, no, CapWT, Solution) :-
    flight(Airport1, Airport3, CarrierNo13, FlightNo13, DepDate13
       , ArrDate13, Duration13, Price13),
```

```
25      Flight1Info=f(Airport1, Airport3, CarrierNo13, FlightNo13,
            DepDate13, ArrDate13, Duration13, Price13),
26      flight(Airport2, Airport4, CarrierNo24, FlightNo24, DepDate24
            , ArrDate24, Duration24, Price24),
27      Flight2Info=f(Airport2, Airport4, CarrierNo24, FlightNo24,
            DepDate24, ArrDate24, Duration24, Price24),
28      avg(Price13,Price24,AvgPrice),
29      avg(Duration13,Duration24,AvgDuration),
30      waitingTime(ArrDate13, ArrDate24, WaitingTime), WaitingTime
            =< CapWT,
31      travelScore(AvgPrice,AvgDuration,WaitingTime,MinP,MaxP,MinD,
            MaxD,CapWT,Rank),
32      Solution=sol(AvgPrice, AvgDuration, WaitingTime, Rank,
            Flight1Info, Flight2Info).
33
34  solutionAndRank(Airport1, Airport2, Airport3, Airport4, MaxP,
        MinP, MaxD, MinD, yes, CapWT, Solution) :-
35      itinerary(Airport1, Airport3, OutboundFlight13,
            ReturnFlight13, Price13),
36      itinerary(Airport2, Airport4, OutboundFlight24,
            ReturnFlight24, Price24),
37      OutboundFlight13 = f(CarrierNo13, FlightNo13, DepDate13,
            ArrDate13, Duration13),
38      OutboundFlight24 = f(CarrierNo24, FlightNo24, DepDate24,
            ArrDate24, Duration24),
39      ReturnFlight13 = f(CarrierNo31, FlightNo31, DepDate31,
            ArrDate31, Duration31),
40      ReturnFlight24 = f(CarrierNo42, FlightNo42, DepDate42,
            ArrDate42, Duration42),
41      avg(Price13, Price24, AvgPrice),
42      avg(Duration13, Duration24, AvgDuration),
43      waitingTime(ArrDate13, ArrDate24, WaitingTime), WaitingTime
            =< CapWT,
44      travelScore(AvgPrice,AvgDuration,WaitingTime,MinP,MaxP,MinD,
            MaxD,CapWT,Rank),
45      Flight1Info = f(Airport1, Airport3, CarrierNo13, FlightNo13,
            DepDate13, ArrDate13, Duration13, Price13),
46      Flight1ReturnInfo = f(Airport3, Airport1, CarrierNo31,
            FlightNo31, DepDate31, ArrDate31, Duration31, Price13),
47      Flight2Info = f(Airport2, Airport4, CarrierNo24, FlightNo24,
            DepDate24, ArrDate24, Duration24, Price24),
48      Flight2ReturnInfo = f(Airport4, Airport2, CarrierNo42,
            FlightNo42, DepDate42, ArrDate42, Duration42, Price24),
49      Solution=sol(AvgPrice, AvgDuration, WaitingTime, Rank,
            Flight1Info, Flight1ReturnInfo, Flight2Info,
            Flight2ReturnInfo).
50
51  eligibleFlights(Airport1, Airport2, MaxP, MinP, MaxD, MinD,
        Return, CapWT, yes, Solution):-
52      airport(Airport3, _, _), dif(Airport1, Airport3), dif(
            Airport2, Airport3),
53      solutionAndRank(Airport1, Airport2, Airport3, Airport3, MaxP,
```

```
        MinP, MaxD, MinD, Return, CapWT, Solution).

eligibleFlights(Airport1, Airport2, MaxP, MinP, MaxD, MinD,
   Return, CapWT, no, Solution):-
    airport(Airport3, City, _), airport(Airport4, City, _),
    solutionAndRank(Airport1, Airport2, Airport3, Airport4, MaxP,
        MinP, MaxD, MinD, Return, CapWT, Solution).

directFlight(Airport1, Airport2, MaxP, MinP, MaxD, MinD, no,
   Solution) :-
    flight(Airport1, Airport2, CarrierNo, FlightNo, DepDate,
       ArrDate, Duration, Price),
    travelScore(Price, Duration, 0, MinP, MaxP, MinD, MaxD, inf,
       Rank),
    Solution=sol(Price, Duration, 0, Rank, f(Airport1, Airport2,
       CarrierNo, FlightNo, DepDate, ArrDate, Duration, Price), _
       ).

directFlight(Airport1, Airport2, MaxP, MinP, MaxD, MinD, yes,
   Solution) :-
    itinerary(Airport1, Airport2, OutboundFlight12,
       ReturnFlight12, Price12),
    OutboundFlight12 = f(CarrierNo12, FlightNo12, DepDate12,
       ArrDate12, Duration12),
    ReturnFlight12 = f(CarrierNo21, FlightNo21, DepDate21,
       ArrDate21, Duration21),
    Flight1Info=f(Airport1, Airport2, CarrierNo12, FlightNo12,
       DepDate12, ArrDate12, Duration12, Price12),
    Flight1ReturnInfo = f(Airport2, Airport1, CarrierNo21,
       FlightNo21, DepDate21, ArrDate21, Duration21, Price12),
    travelScore(Price12,Duration12,0,MinP,MaxP,MinD,MaxD,inf,Rank
       ),
    Solution=sol(Price12, Duration12, 0, Rank, Flight1Info,
       Flight1ReturnInfo, _, _).

% normalize the values to get a travelScore between 0 - 1
travelScore(AvgPrice, AvgDuration, WaitingTime, MinP, MaxP, MinD,
    MaxD, CapWT, Rank) :-
    NormalizedPrice is (AvgPrice - MinP) / (MaxP - MinP),
    NormalizedDuration is (AvgDuration - MinD) / (MaxD - MinD),
    NormalizedWaitingTime is (WaitingTime) / (CapWT),
    wPrice(AP), wDuration(AD), wWaitingTime(AWT),
    Rank is AP*NormalizedPrice + AD*NormalizedDuration + AWT*
       NormalizedWaitingTime.

sortByStrategy(L, price, SortedL) :- sort(1, @=<, L, SortedL).
sortByStrategy(L, duration, SortedL) :- sort(2, @=<, L, SortedL).
sortByStrategy(L, waitingtime, SortedL) :- sort(3, @=<, L,
   SortedL).
sortByStrategy(L, bestsolution, SortedL) :- sort(4, @=<, L,
   SortedL).

```

```
% Predicate to find the max, min and avg of two numbers
max(A, B, Max) :- A >= B, Max is A.
max(A, B, Max) :- A < B, Max is B.

min(A, B, Min) :- A =< B, Min is A.
min(A, B, Min) :- A > B, Min is B.

avg(X,Y,A) :- A is (X+Y)/2.

% Predicate to find the maximum and minimum in a list of numbers
min_max_list([H|T], Min, Max) :- min_max_list(T, H, H, Min, Max).
min_max_list([], Min, Max, Min, Max).
min_max_list([H|T], CurrMin, CurrMax, Min, Max) :-
    min(H, CurrMin, NewMin), max(H, CurrMax, NewMax),
    min_max_list(T, NewMin, NewMax, Min, Max).

waitingTime(ArrDate13, ArrDate23, WaitingTime) :-
    date_time_stamp(ArrDate13, ArrDate13ts),
    date_time_stamp(ArrDate23, ArrDate23ts),
    WaitingTime is abs(ArrDate23ts - ArrDate13ts).

minAndMax(What, Return, Max, Min) :-
    getAll(What, Return, All),
    min_max_list(All, Min, Max).

getAll(price, no, Prices) :- findall(Price, flight(_, _, _, _, _,
    _, _, Price), Prices).
getAll(duration, no, Durations) :- findall(Duration, flight(_, _,
    _, _, _, _, Duration, _), Durations).

getAll(price, yes, Prices) :- findall(Price, itinerary(_, _, _, _
   , Price), Prices).
getAll(duration, yes, Durations) :- findall(Duration, itinerary(_
   , _, f(_,_,_,_,Duration), _, _), Durations).
```

Bibliografia

[1] William F Clocksin e Christopher S Mellish. *Programming in PROLOG*. Springer Science & Business Media, 2003.

[2] Mónica Edwards-Schachter. «The nature and variety of innovation». In: *International Journal of Innovation Studies* 2.2 (2018), pp. 65–79. ISSN: 2096-2487. DOI: https://doi.org/10.1016/j.ijis.2018.08.004. URL: https://www.sciencedirect.com/science/article/pii/S2096248718300249.

[3] Jean H Gallier e Stan Raatz. «SLD-Resolution Methods for Horn Clauses with Equality Based on E-Unification.» In: *SLP*. Vol. 86. 1986, pp. 168–179.

[4] Miguel Grinberg. *Flask web development: developing web applications with python*. " O'Reilly Media, Inc.", 2018.

[5] Riccardo Guidotti et al. «A survey of methods for explaining black box models». In: *ACM computing surveys (CSUR)* 51.5 (2018), pp. 1–42.

[6] Robert Kowalski e Keith L Clark. «Logic programming». In: *Encyclopedia of Computer Science*. 2003, pp. 1017–1031.

[7] Ronny Lempel et al. «Just in time indexing for up to the second search». In: *Proceedings of the sixteenth ACM conference on Conference on Information and Knowledge Management*. 2007, pp. 97–106.

[8] Mark Lutz. *Programming python*. " O'Reilly Media, Inc.", 2001.

[9] Donald A Norman. *The psychology of everyday things*. Basic books, 1988.

[10] Vijay Surwase. «REST API modeling languages-a developer's perspective». In: *Int. J. Sci. Technol. Eng* 2.10 (2016), pp. 634–637.